



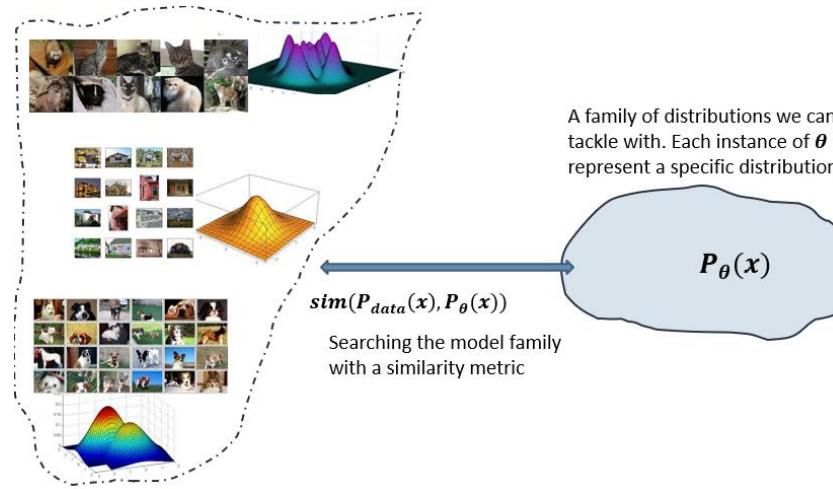
Score-based Models

22-808: Generative models
Sharif University of Technology
Fall 2025

Fatemeh Seyyedsalehi

Slides of this lecture are selected from Yong Song's and Stanford CS236

Recap



- ▶ We need a framework to interact with distributions for statistical generative models.
 - ▶ Probabilistic generative models
 - ▶ Deep generative models
 - ▶ Autoregressive models
 - ▶ Variational Autoencoders
 - ▶ Generative adversarial networks
 - ▶ Normalizing Flow
 - ▶ Energy-based models
 - ▶ **Score-based models**

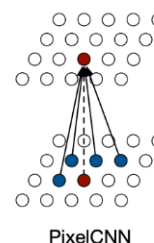
How to represent probability distributions?

- Probability density function (p.d.f.) or probability mass function (p.m.f.)

$$p(\mathbf{x})$$

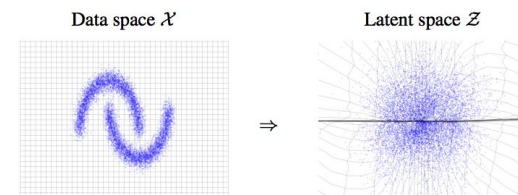
- Autoregressive models

$$p_{\theta}(\mathbf{x}) = \prod_{i=1}^d p_{\theta}(\mathbf{x}_i \mid \mathbf{x}_{<i})$$



- Flow models

$$p_{\theta}(\mathbf{x}) = p(\mathbf{z}) |\det(J_{f_{\theta}}(\mathbf{x}))|, \quad \mathbf{z} = f_{\theta}(\mathbf{x})$$



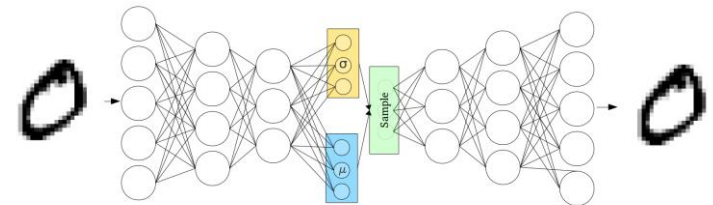
How to represent probability distributions?

- Probability density function (p.d.f.) or probability mass function (p.m.f.)

$$p(\mathbf{x})$$

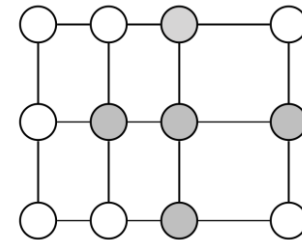
- Variational autoencoders

$$p_{\theta}(\mathbf{x}) = \int p(\mathbf{z})p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z}$$



- Energy-based models

$$p_{\theta}(\mathbf{x}) = \frac{e^{f_{\theta}(\mathbf{x})}}{Z(\theta)}$$



How to represent probability distributions?

- Probability density function (p.d.f.) or probability mass function (p.m.f.)

$$p(\mathbf{x})$$

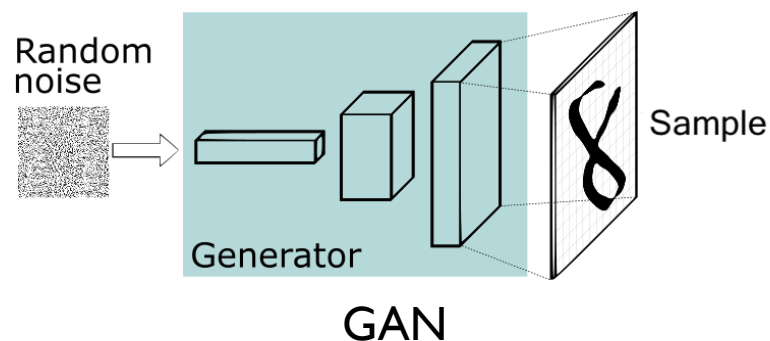
- Pros
 - Maximum likelihood training
 - Principled model comparison via likelihoods
- Cons
 - Special architectures or surrogate losses to deal with intractable partition functions



How to represent probability distributions?

- Sampling process
- Generative adversarial networks (GANs)

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} = g_{\theta}(\mathbf{z})$$



How to represent probability distributions?

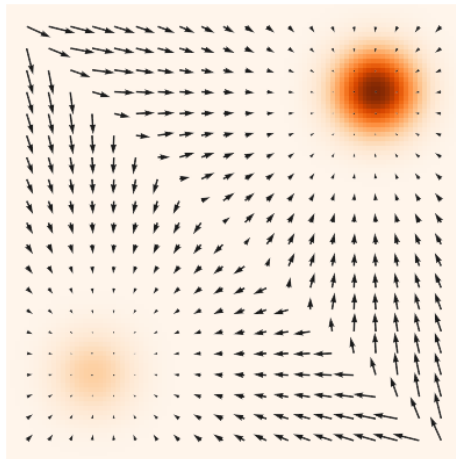
- Sampling process
- Pros
 - Samples typically have better quality
- Cons
 - Require adversarial training. Training instability and mode collapse.
 - No principled way to compare different models
 - No principled termination criteria for training



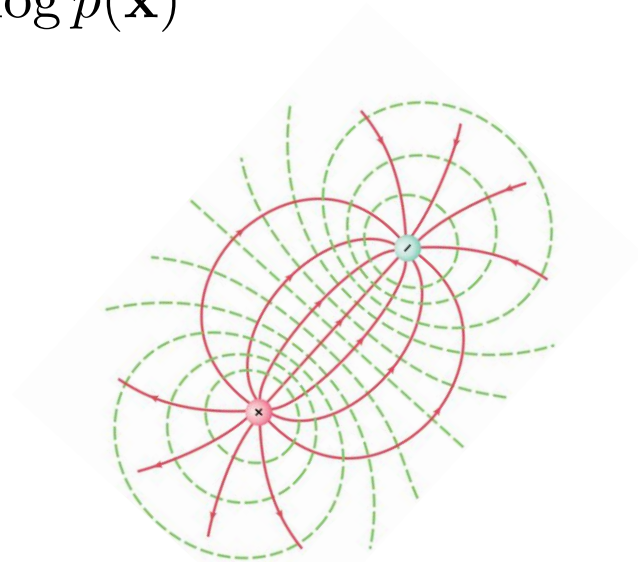
How to represent probability distributions?

- When the pdf is differentiable, we can compute the gradient of a probability density.

Score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$



(pdf and score)

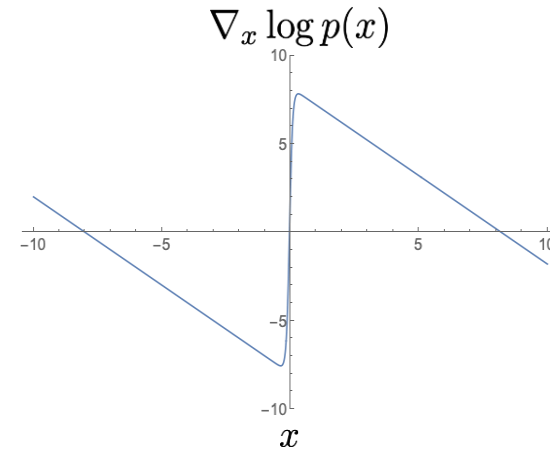
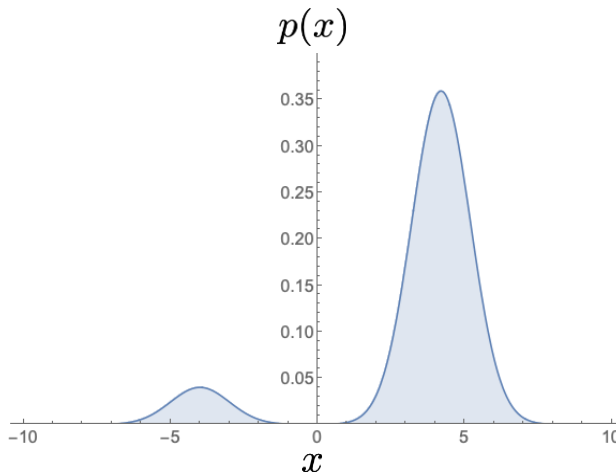


(Electrical potentials and fields)

How to represent probability distributions?

- When the pdf is differentiable, we can compute the gradient of a probability density.

Score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$



Recap on energy-based models

- Deep Energy-Based models (EBMs)

$$f_{\theta}(\mathbf{x}) \in \mathbb{R}$$

$$p_{\theta}(\mathbf{x}) = \frac{e^{f_{\theta}(\mathbf{x})}}{Z(\theta)}$$



- Maximum likelihood training: $\max_{\theta} f_{\theta}(\mathbf{x}_{\text{train}}) - \log Z(\theta)$
 - Contrastive divergence

$$\nabla_{\theta} f_{\theta}(\mathbf{x}_{\text{train}}) - \nabla_{\theta} \log Z(\theta) \approx \nabla_{\theta} f_{\theta}(\mathbf{x}_{\text{train}}) - \nabla_{\theta} f_{\theta}(\mathbf{x}_{\text{sample}})$$

- Requires iterative sampling during training

$$\mathbf{x}_{\text{sample}} \sim p_{\theta}(\mathbf{x})$$

Recap on energy-based models

- Deep Energy-Based models (EBMs)

$$f_{\theta}(\mathbf{x}) \in \mathbb{R}$$

$$p_{\theta}(\mathbf{x}) = \frac{e^{f_{\theta}(\mathbf{x})}}{Z(\theta)}$$



- Minimizing Fisher divergence:

$$\min_{\theta} \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2]$$

- Score matching

$$\begin{aligned} & \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2] \\ &= \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2 + \text{tr}(\nabla_{\mathbf{x}}^2 \log p_{\theta}(\mathbf{x})) \right] + \text{const.} \end{aligned}$$

Score matching for training EBM

- Score function of EBM

$$\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z(\theta)}_{=0} = \nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$$

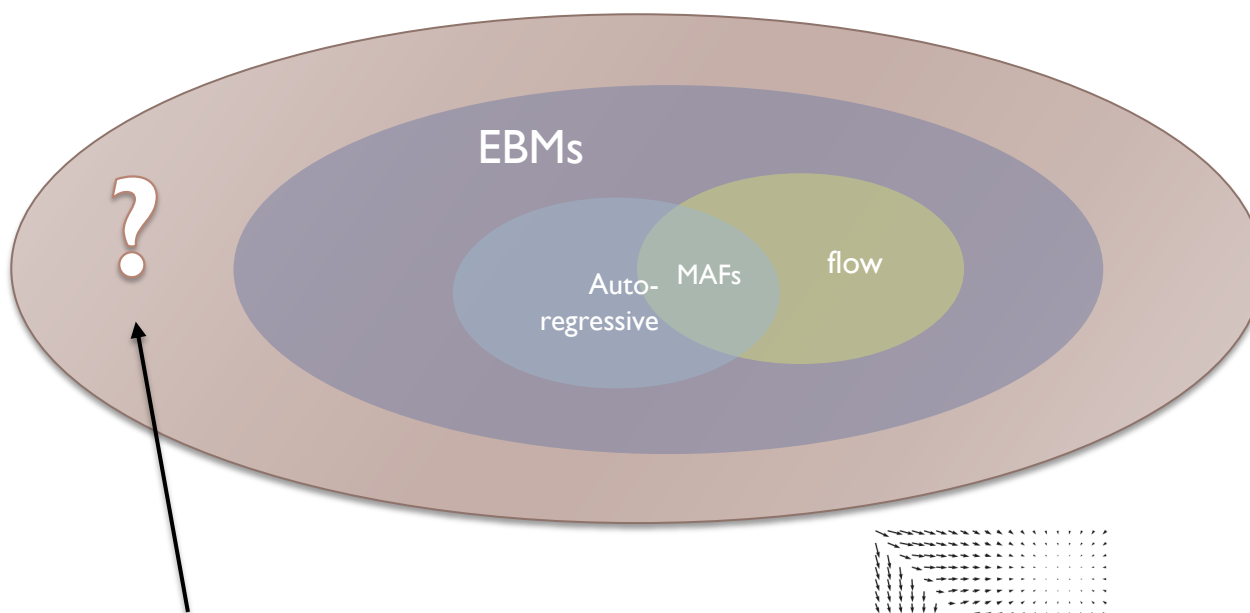
- Score matching for EBM:

$$\begin{aligned} & E_{\mathbf{x} \sim p_{\text{data}}} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2 + \text{tr}(\nabla_{\mathbf{x}}^2 \log p_{\theta}(\mathbf{x})) \right] \\ &= E_{\mathbf{x} \sim p_{\text{data}}} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})\|_2^2 + \text{tr}(\nabla_{\mathbf{x}}^2 f_{\theta}(\mathbf{x})) \right] \end{aligned}$$

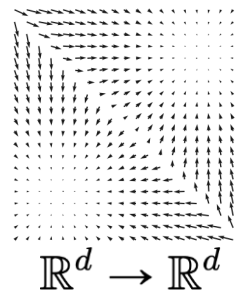
- Is score matching limited to EBM?
 - Autoregressive models
 - Normalizing flow models

Score-based models

- What's the most general model that can be efficiently trained by score matching?



$$\mathbf{s}_\theta(\mathbf{x})$$

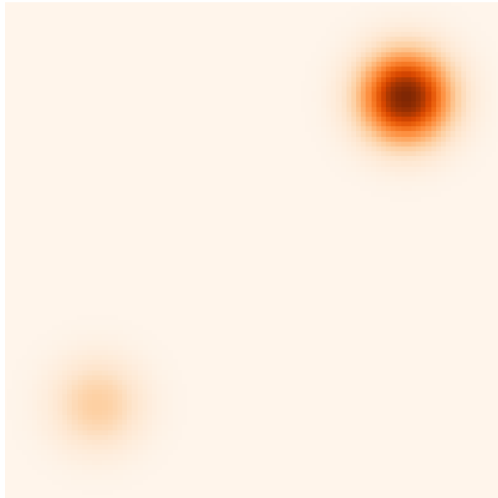


Directly model
the vector field
of gradients!

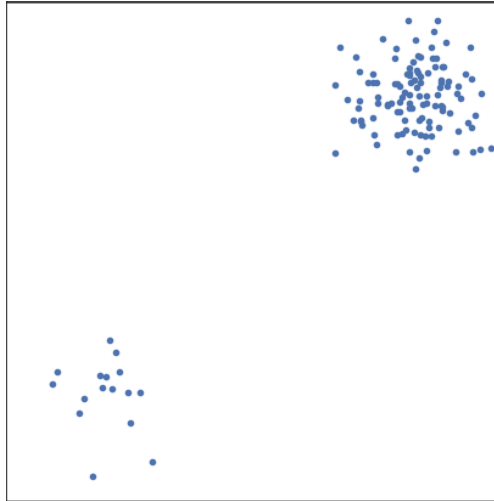
- Score-based model

Score estimation by training score-based models

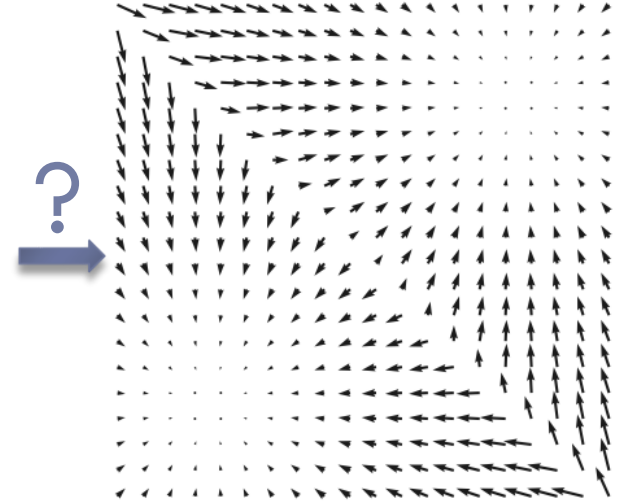
Probability density
 $p_{\text{data}}(\mathbf{x})$



i.i.d. samples
 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$

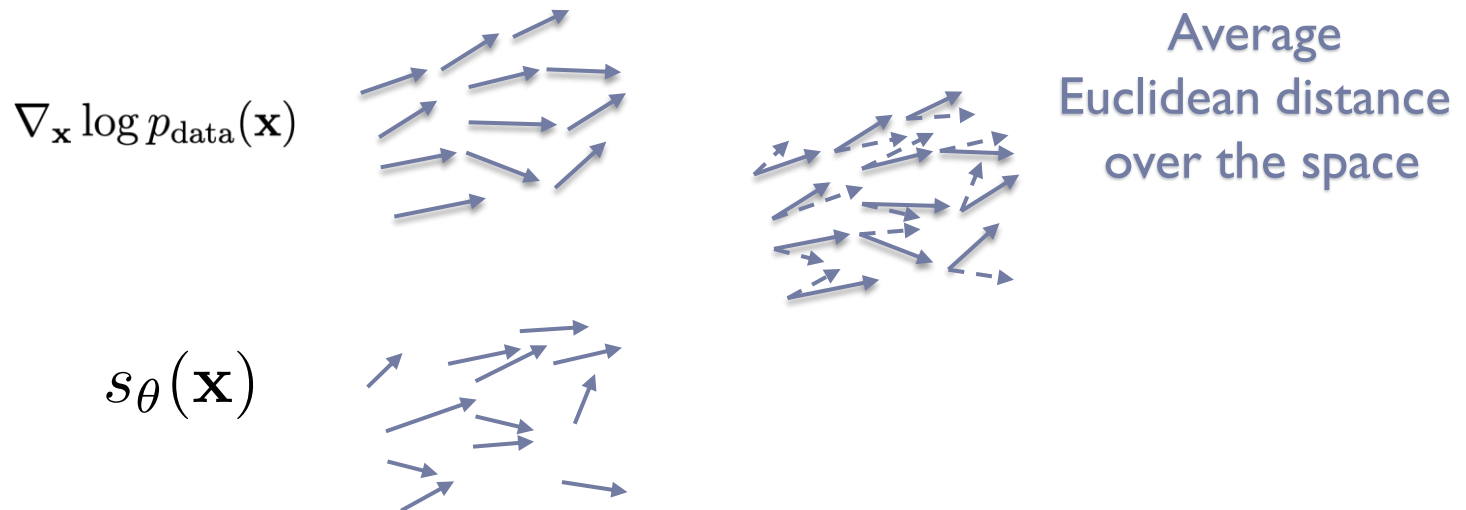


Score function
 $\mathbf{s}_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$



Score estimation by training score-based models

- **Given:** i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- **Task:** Estimating the score $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$
- **Score Model:** A learnable vector-valued function
- **Goal:** $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$ $s_{\theta}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d$
- How to compare two vector fields of scores?



Score estimation by training score-based models

- **Objective:** Average Euclidean distance over the whole space.

$$\frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2]$$

(Fisher divergence)

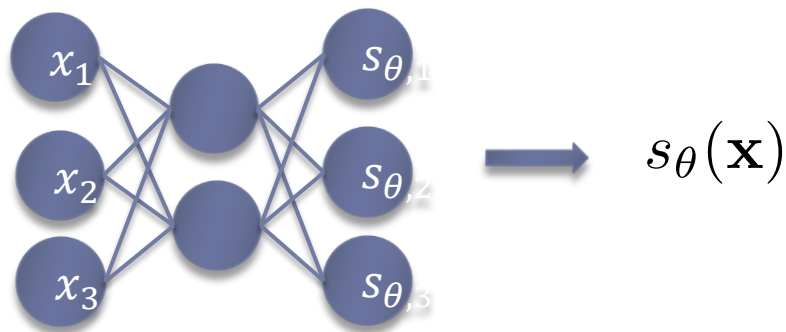
- **Score matching:**

$$E_{\mathbf{x} \sim p_{\text{data}}} \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 + \text{tr} \left(\underbrace{\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})}_{\text{Jacobian of } \mathbf{s}_{\theta}(\mathbf{x})} \right) \right]$$

- **Requirements:**
 - The score model must be efficient to evaluate.
 - Do we need the score model to be a proper score function (i.e., gradient of a scalar “energy” function)?

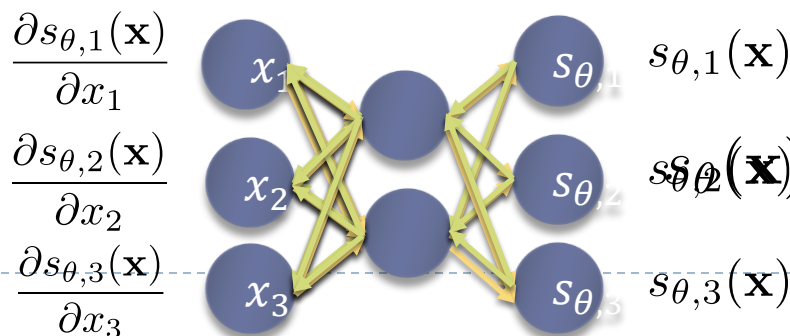
Score matching is not scalable

- Deep neural networks as more expressive score models



Score Matching
is not Scalable!

- Compute $\|s_{\theta}(\mathbf{x})\|_2^2$ and $\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}))$ ☹️



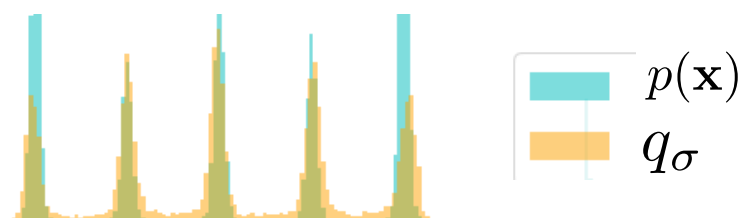
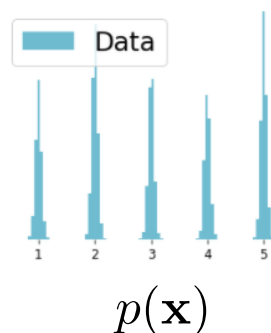
$O(d)$ Backprops!

$$\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) = \begin{pmatrix} \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_3} \end{pmatrix}$$

Denoising Score Matching (Vincent, 2011)

- Consider the perturbed distribution

$$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}; \sigma^2 I) \quad q_{\sigma}(\tilde{\mathbf{x}}) = \int p(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x}$$



- Score estimation for $\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})$ is easier $q_{\sigma}(\tilde{\mathbf{x}}) \approx p(\tilde{\mathbf{x}})$
- If the noise level is small, this is a good approximation

Denoising score matching



$\mathbf{x}$

$p_{\text{data}}(\mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}})$



$\tilde{\mathbf{x}}$

Denoising score matching (Vincent 2011):

- matching the score of a noise-perturbed distribution

$$\begin{aligned}
 & \frac{1}{2} E_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2] \\
 &= \frac{1}{2} \int q_{\sigma}(\tilde{\mathbf{x}}) \|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2 d\tilde{\mathbf{x}} \\
 &= \frac{1}{2} \int q_{\sigma}(\tilde{\mathbf{x}}) \|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})\|_2^2 d\tilde{\mathbf{x}} + \frac{1}{2} \int q_{\sigma}(\tilde{\mathbf{x}}) \|\mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2 d\tilde{\mathbf{x}} \\
 &\quad - \int q_{\sigma}(\tilde{\mathbf{x}}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= \text{const.} + \frac{1}{2} E_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2] - \int q_{\sigma}(\tilde{\mathbf{x}}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}}
 \end{aligned}$$

Denoising score matching



$\mathbf{x}$

$p_{\text{data}}(\mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}})$



$\tilde{\mathbf{x}}$

$$\begin{aligned}
 & - \int q_{\sigma}(\tilde{\mathbf{x}}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \int q_{\sigma}(\tilde{\mathbf{x}}) \frac{1}{q_{\sigma}(\tilde{\mathbf{x}})} \nabla_{\tilde{\mathbf{x}}} q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \int \nabla_{\tilde{\mathbf{x}}} q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \int \nabla_{\tilde{\mathbf{x}}} \left(\int p_{\text{data}}(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x} \right)^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \int \left(\int p_{\text{data}}(\mathbf{x}) \nabla_{\tilde{\mathbf{x}}} q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x} \right)^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \int \left(\int p_{\text{data}}(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x} \right)^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= - \iint p_{\text{data}}(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\mathbf{x} d\tilde{\mathbf{x}} \\
 &= - E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})} [\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}})]
 \end{aligned}$$

Denoising score matching



$\mathbf{x}$

$p_{\text{data}}(\mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})$

$q_{\sigma}(\tilde{\mathbf{x}})$



$\tilde{\mathbf{x}}$

$$\begin{aligned}
 & \frac{1}{2} E_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2] \\
 &= \text{const.} + \frac{1}{2} E_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2] - \int q_{\sigma}(\tilde{\mathbf{x}}) \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}} \\
 &= \text{const.} + \frac{1}{2} E_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2] - E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})^{\top} \mathbf{s}_{\theta}(\tilde{\mathbf{x}})] \\
 &= \text{const.} + \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2] \\
 &\quad - \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2] \\
 &= \text{const.} + \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2] + \text{const.} \\
 &= \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2] + \text{const.}
 \end{aligned}$$

Denoising score matching

- Estimate the score of a noise-perturbed distribution

$$\begin{aligned} & \frac{1}{2} E_{\tilde{\mathbf{x}} \sim p_{\text{data}}} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})\|_2^2] \\ &= \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2] + \text{const.} \end{aligned}$$

- $\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})$ is easy to compute
 - $q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}} | \mathbf{x}, \sigma^2 \mathbf{I})$
 - $\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2}$
- **Pros:** efficient to optimize even for very high dimensional data, and useful for optimal denoising.
- **Con:** cannot estimate the score of clean data (noise-free)

Denoising score matching

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of perturbed datapoints

$$\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_n\} \sim q_{\sigma}(\tilde{\mathbf{x}})$$

- Estimate the denoising score matching loss with empirical means

$$\tilde{\mathbf{x}}_i \sim q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)$$
$$\frac{1}{2n} \sum_{i=1}^n [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)\|_2^2]$$

- If Gaussian perturbation

$$\frac{1}{2n} \sum_{i=1}^n \left[\left\| \mathbf{s}_{\theta}(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2} \right\|_2^2 \right]$$

- Stochastic gradient descent
- Need to choose a very small σ !

Denoising Score Matching (Vincent, 2011)

- Consider the perturbed distribution

$$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}; \sigma^2 I) \quad q_{\sigma}(\tilde{\mathbf{x}}) = \int p(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x}$$

- Score estimation for $\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})$ is easier

Score matching $\frac{1}{2} \mathbb{E}_{q_{\sigma}(\tilde{\mathbf{x}})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - s_{\theta}(\tilde{\mathbf{x}})\|_2^2]$

Denoising $= \frac{1}{2} \mathbb{E}_{p(\mathbf{x})} \mathbb{E}_{q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\| \underbrace{\frac{1}{\sigma^2}(\mathbf{x} - \tilde{\mathbf{x}})}_{s_{\theta}(\tilde{\mathbf{x}})} - s_{\theta}(\tilde{\mathbf{x}}) \|_2^2] + \text{const.}$



$\mathbf{x}$

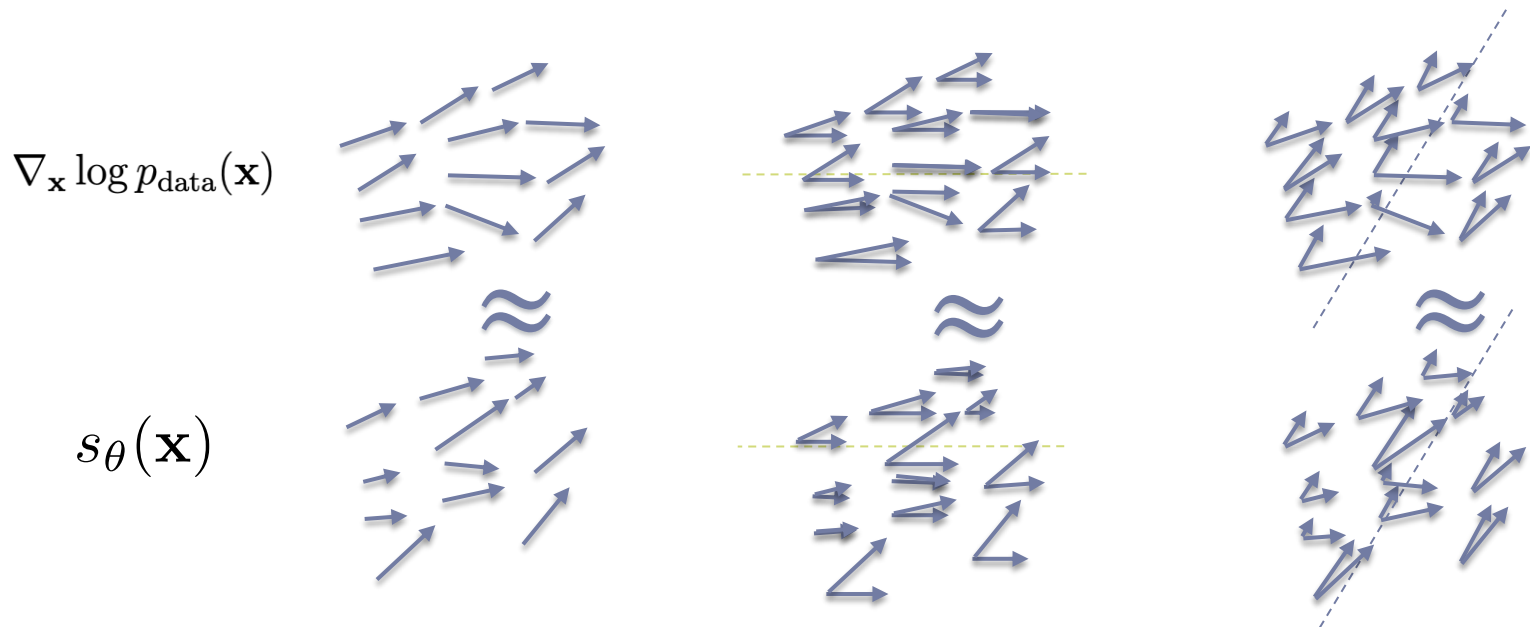


$\tilde{\mathbf{x}} = \mathbf{x} + \text{noise}$

$s_{\theta}(\tilde{\mathbf{x}})$ tries to estimate the noise that was added to produce $\tilde{\mathbf{x}}$

Sliced score matching

- One dimensional problems should be easier.
- Consider projections onto random directions.



Song*, Garg*, Shi, Ermon. "Sliced Score Matching: A Scalable Approach to Density and Score Estimation." UAI 2019.

Sliced score matching

- **Objective:** Sliced Fisher Divergence

$$\frac{1}{2} E_{\mathbf{v} \sim p_{\mathbf{v}}} E_{\mathbf{x} \sim p_{\text{data}}} [(\mathbf{v}^T \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{v}^T \mathbf{s}_{\theta}(\mathbf{x}))^2]$$

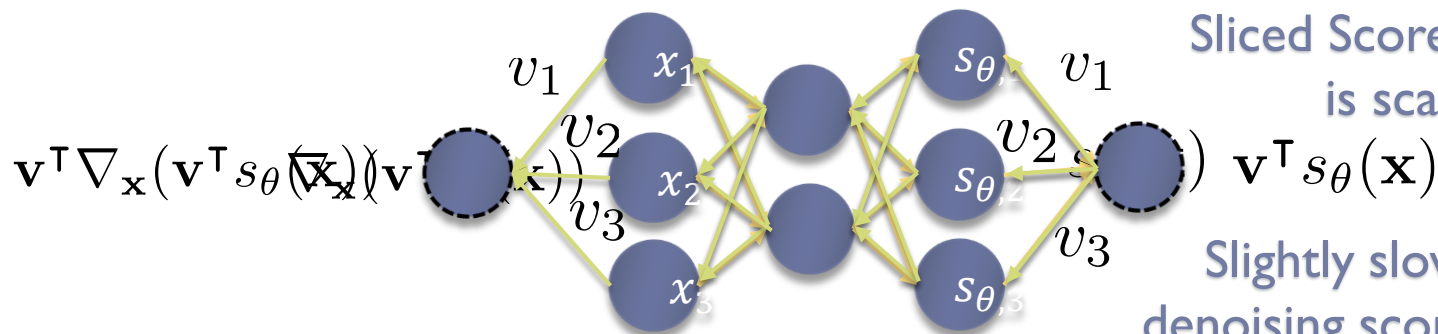
- **Integration by parts**

$$E_{\mathbf{v} \sim p_{\mathbf{v}}} E_{\mathbf{x} \sim p_{\text{data}}} \left[\mathbf{v}^T \nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x}) \mathbf{v} + \frac{1}{2} (\mathbf{v}^T \mathbf{s}_{\theta}(\mathbf{x}))^2 \right]$$

$$\begin{pmatrix} v_1 & v_2 & v_3 \end{pmatrix} \begin{pmatrix} \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,1}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,2}(\mathbf{x})}{\partial x_3} \\ \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_1} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_2} & \frac{\partial s_{\theta,3}(\mathbf{x})}{\partial x_3} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

Computing Jacobian-vector products is scalable

$$\mathbf{v}^\top \nabla_{\mathbf{x}} s_\theta(\mathbf{x}) \mathbf{v} = \mathbf{v}^\top \nabla_{\mathbf{x}} (\mathbf{v}^\top s_\theta(\mathbf{x}))$$



One Backprop!
Sliced Score Matching
is scalable

Slightly slower than
denoising score matching

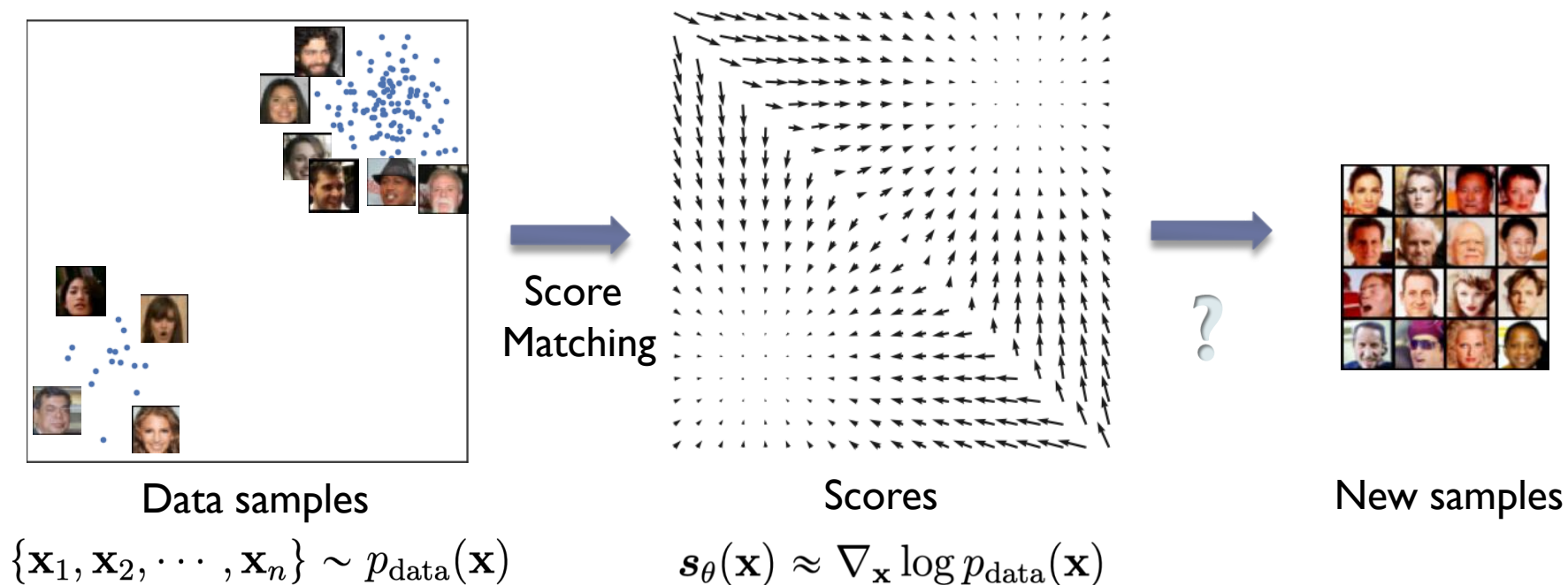
Sliced score matching

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of projection directions $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\} \sim p_{\mathbf{v}}$
- Estimate the sliced score matching loss with empirical means

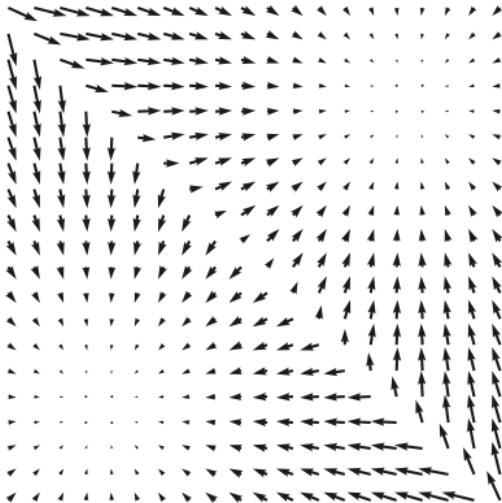
$$\frac{1}{n} \sum_{i=1}^n \left[\mathbf{v}_i^{\top} \nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x}_i) \mathbf{v}_i + \frac{1}{2} (\mathbf{v}_i^{\top} \mathbf{s}_{\theta}(\mathbf{x}_i))^2 \right]$$

- The projection distribution is typically Gaussian or Rademacher
- Stochastic gradient descent
- Can use more projections per datapoint to boost performance

Score-based generative modeling

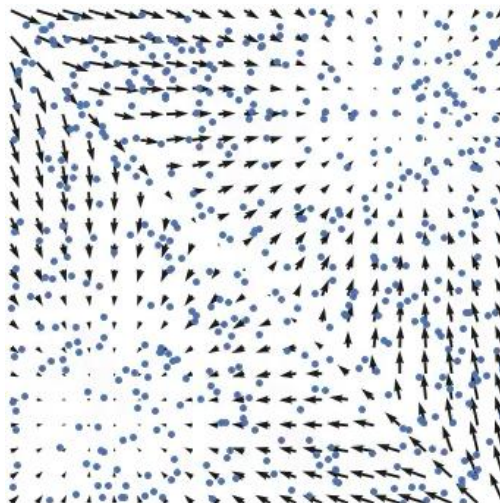


From scores to samples: Langevin MCMC



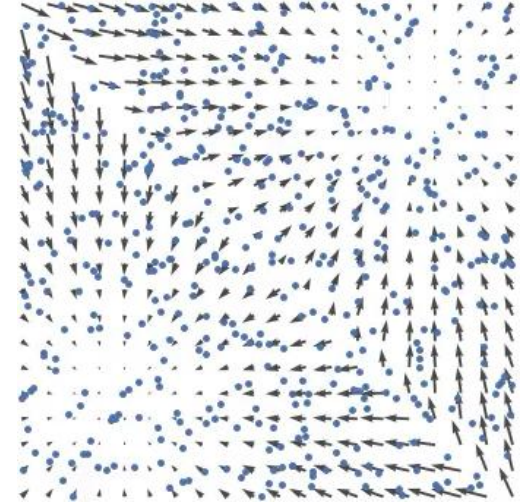
Scores

$$\mathbf{s}_\theta(\mathbf{x})$$



Follow the scores

$$\tilde{\mathbf{x}}_{t+1} \leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2} \mathbf{s}_\theta(\tilde{\mathbf{x}}_t)$$



Follow noisy scores:
Langevin MCMC

$$\begin{aligned} \mathbf{z}_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ \tilde{\mathbf{x}}_{t+1} &\leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2} \mathbf{s}_\theta(\tilde{\mathbf{x}}_t) + \sqrt{\epsilon} \mathbf{z}_t \end{aligned}$$

Langevin dynamics sampling

Sample from $p(\mathbf{x})$ using only the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$

- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
- Repeat for $t \leftarrow 1, 2, \dots, T$

$$\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon} \mathbf{z}^t$$

- If $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, we are guaranteed to have

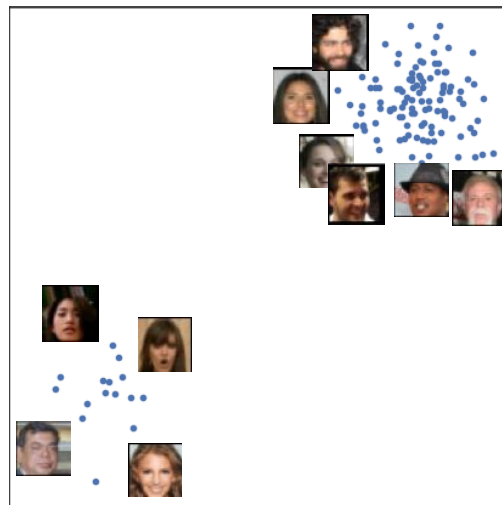
$$\mathbf{x}^T \sim p(\mathbf{x})$$

- Langevin dynamics + score estimation

$$\mathbf{s}_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$



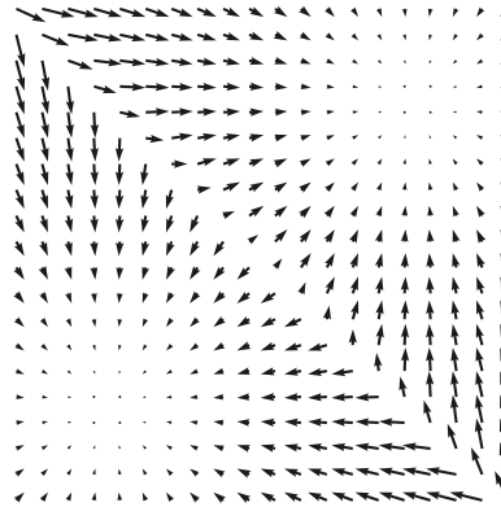
Score-based generative modeling



Data samples

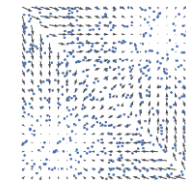
$$\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$$

score
matching



Scores

$$\mathbf{s}_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$$

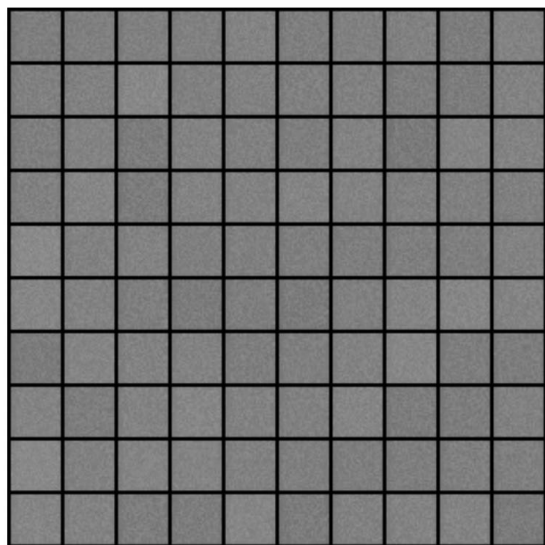


Langevin
dynamics



New samples

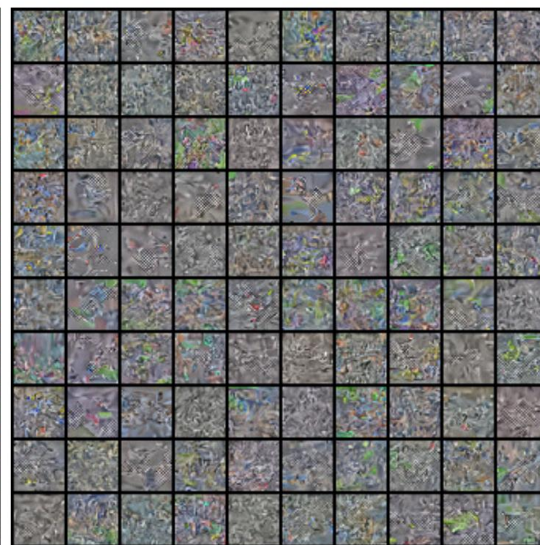
Score-based generative modeling: results



(a) MNIST



(b) CelebA

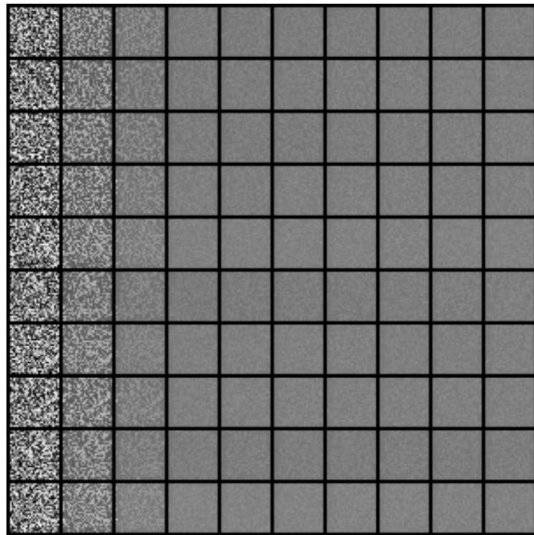


(c) CIFAR-10

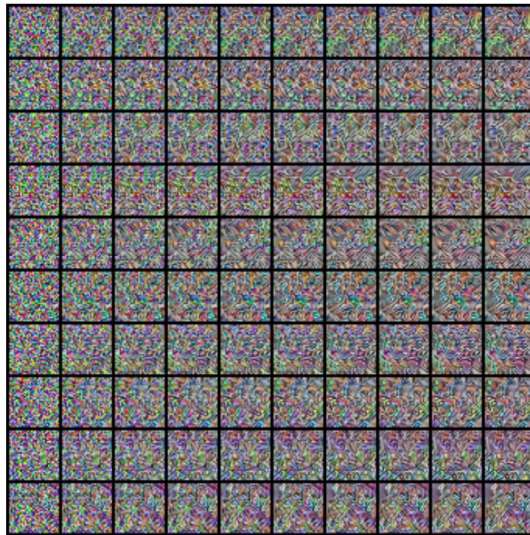
Final samples



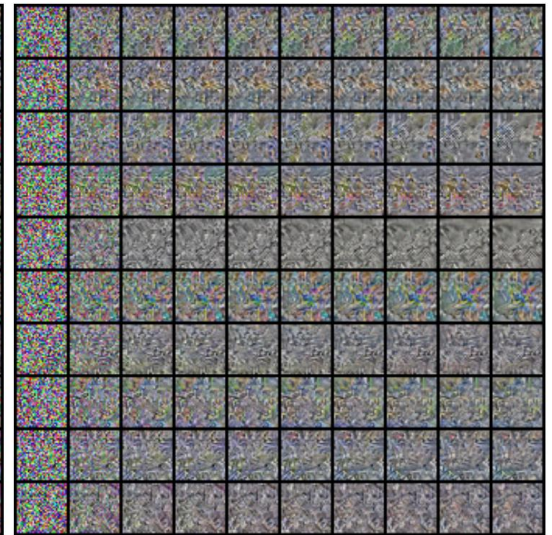
Score-based generative modeling: results



(a) MNIST



(b) CelebA



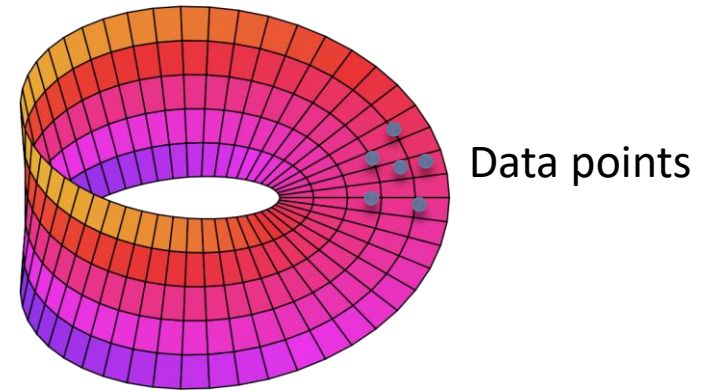
(c) CIFAR-10

Langevin sampling process



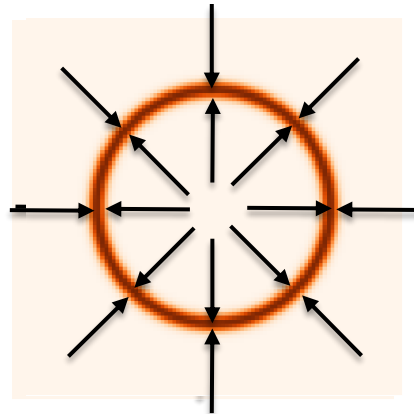
Pitfall 1: manifold hypothesis

- Manifold hypothesis.



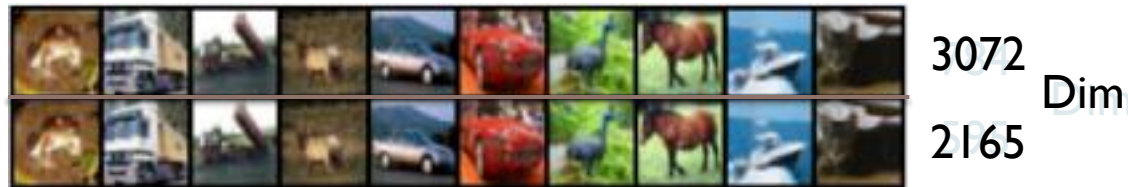
- Data score is undefined.

$$\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$$

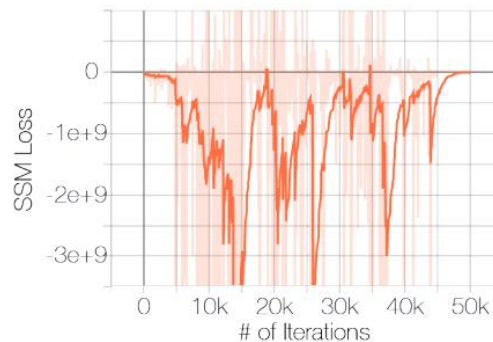


Pitfall 1: manifold hypothesis

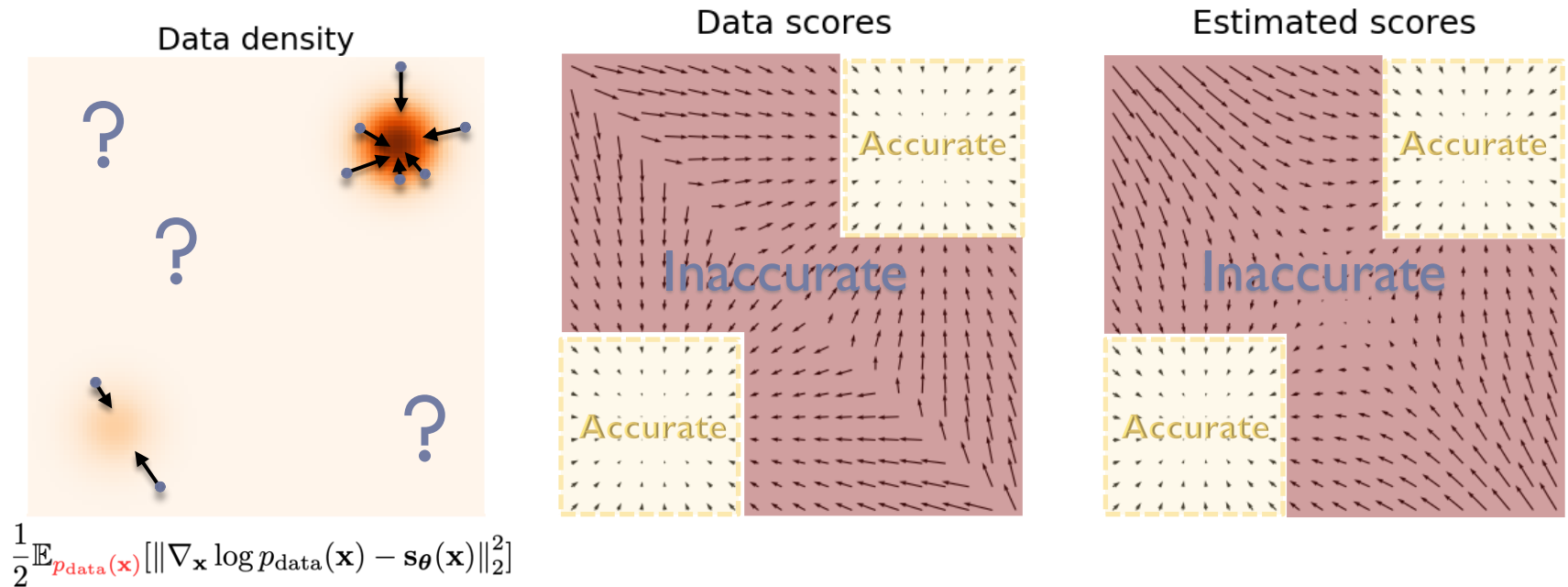
- Fitting the data with a low-dimensional linear manifold (PCA)



- Score estimation on CIFAR-10.



Challenge in low data density regions



**Langevin MCMC will have trouble
exploring low density regions**

Song and Ermon. "Generative Modeling by Estimating Gradients
of the Data Distribution." NeurIPS 2019.

Pitfall 3: slow mixing of Langevin dynamics between data modes

- Suppose the data distribution has two modes with disjoint supports:

$$p_{\text{data}}(\mathbf{x}) = \pi p_1(\mathbf{x}) + (1 - \pi)p_2(\mathbf{x})$$

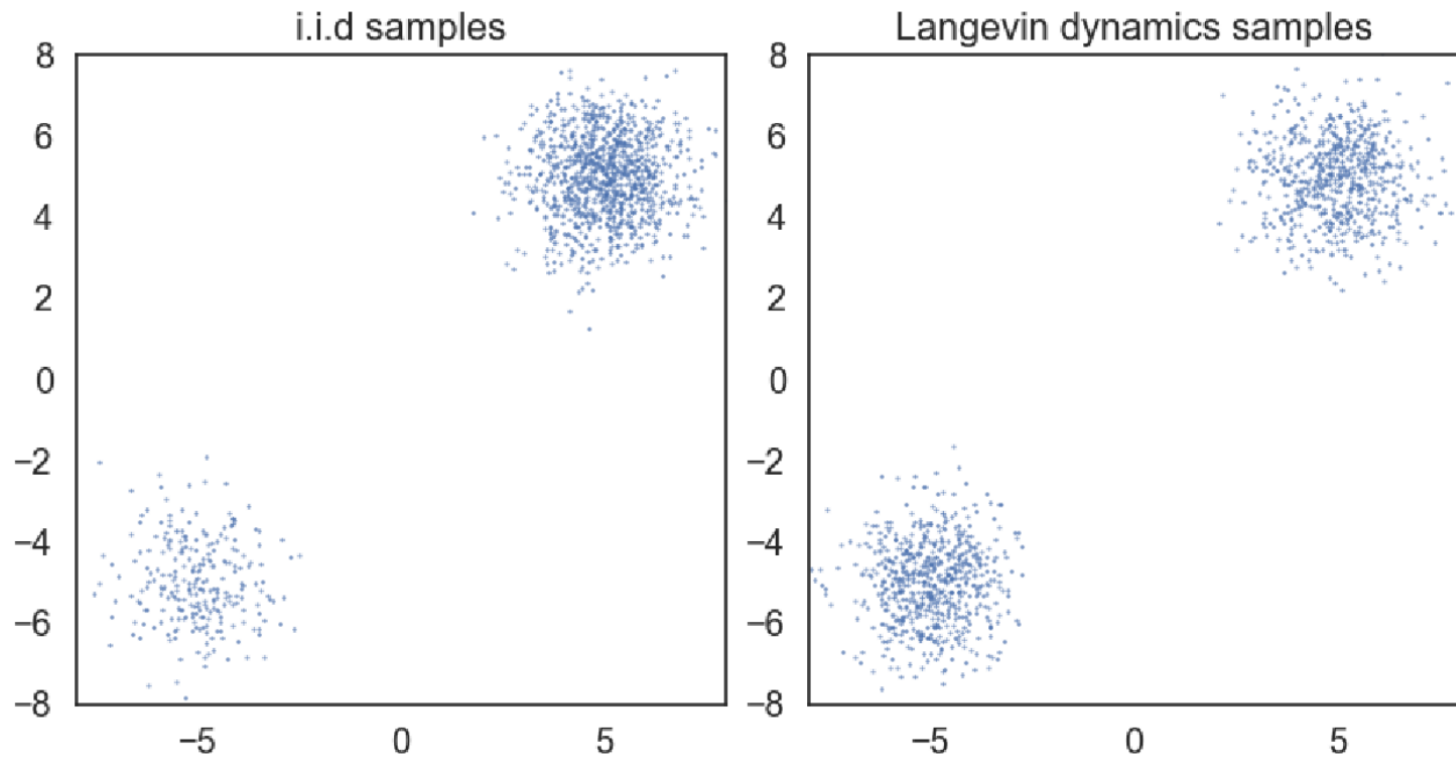
$$\mathcal{A} \cap \mathcal{B} = \emptyset \quad p_{\text{data}}(\mathbf{x}) = \begin{cases} \pi p_1(\mathbf{x}), & \mathbf{x} \in \mathcal{A} \\ (1 - \pi)p_2(\mathbf{x}), & \mathbf{x} \in \mathcal{B} \end{cases}$$

- Data score function:

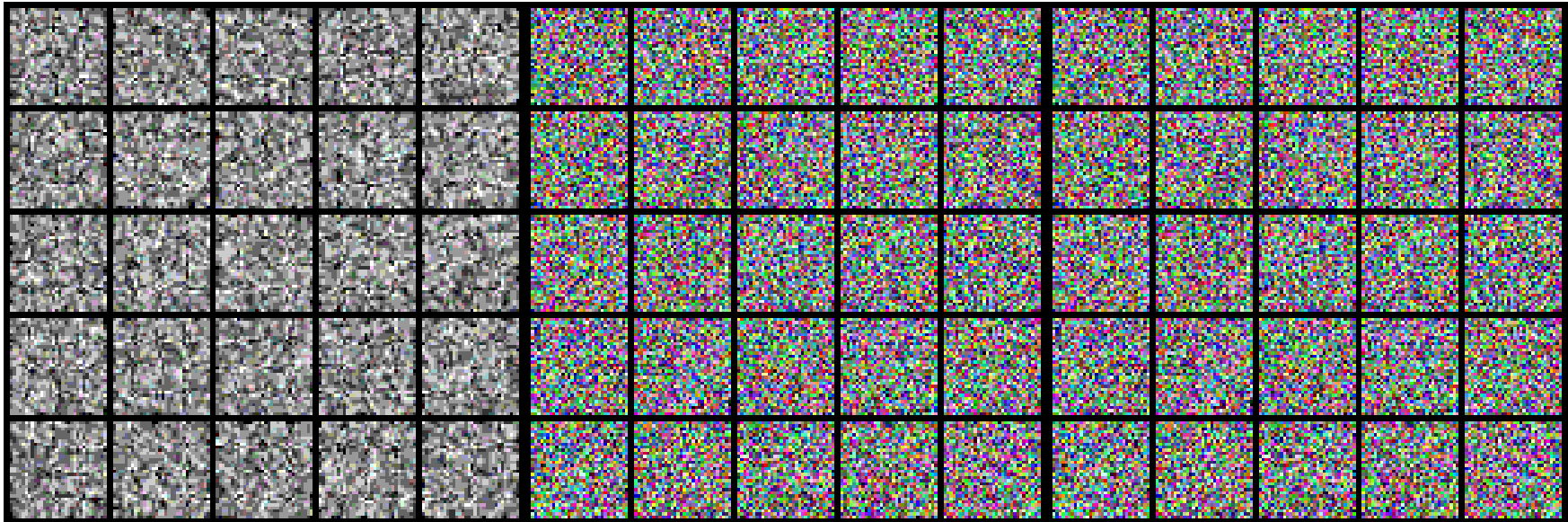
$$\begin{aligned} \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) &= \begin{cases} \nabla_{\mathbf{x}} [\log \pi + \log p_1(\mathbf{x})], & \mathbf{x} \in \mathcal{A} \\ \nabla_{\mathbf{x}} [\log(1 - \pi) + \log p_2(\mathbf{x})], & \mathbf{x} \in \mathcal{B} \end{cases} \\ &= \begin{cases} \nabla_{\mathbf{x}} \log p_1(\mathbf{x}), & \mathbf{x} \in \mathcal{A} \\ \nabla_{\mathbf{x}} \log p_2(\mathbf{x}), & \mathbf{x} \in \mathcal{B} \end{cases} \end{aligned}$$

- The score function has no dependence on the mode weighting π at all!
- Langevin sampling will not reflect π

Pitfall 3: slow mixing of Langevin dynamics between data modes



After fixing these pitfalls

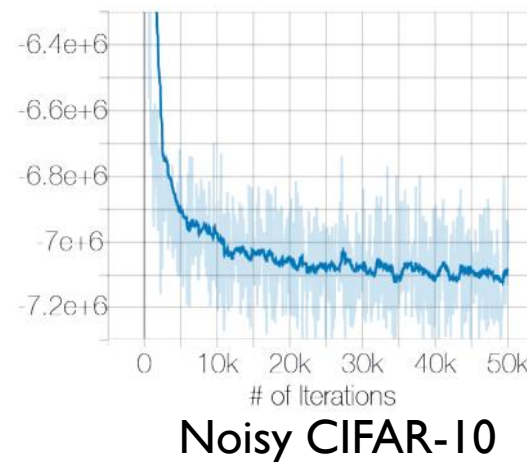
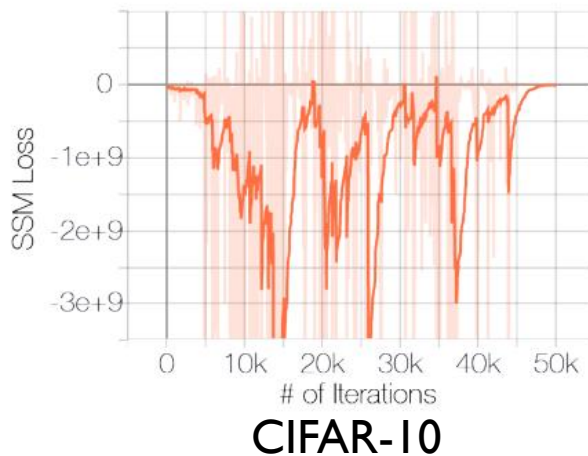
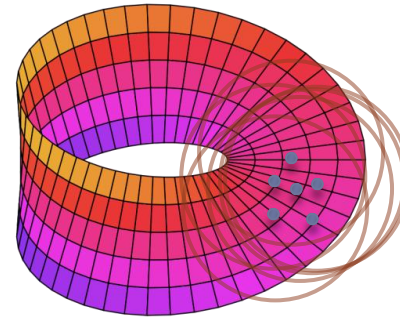


Song, Yang, and Stefano Ermon. "Generative Modeling
by Estimating Gradients of the Data Distribution."
NeurIPS 2019.



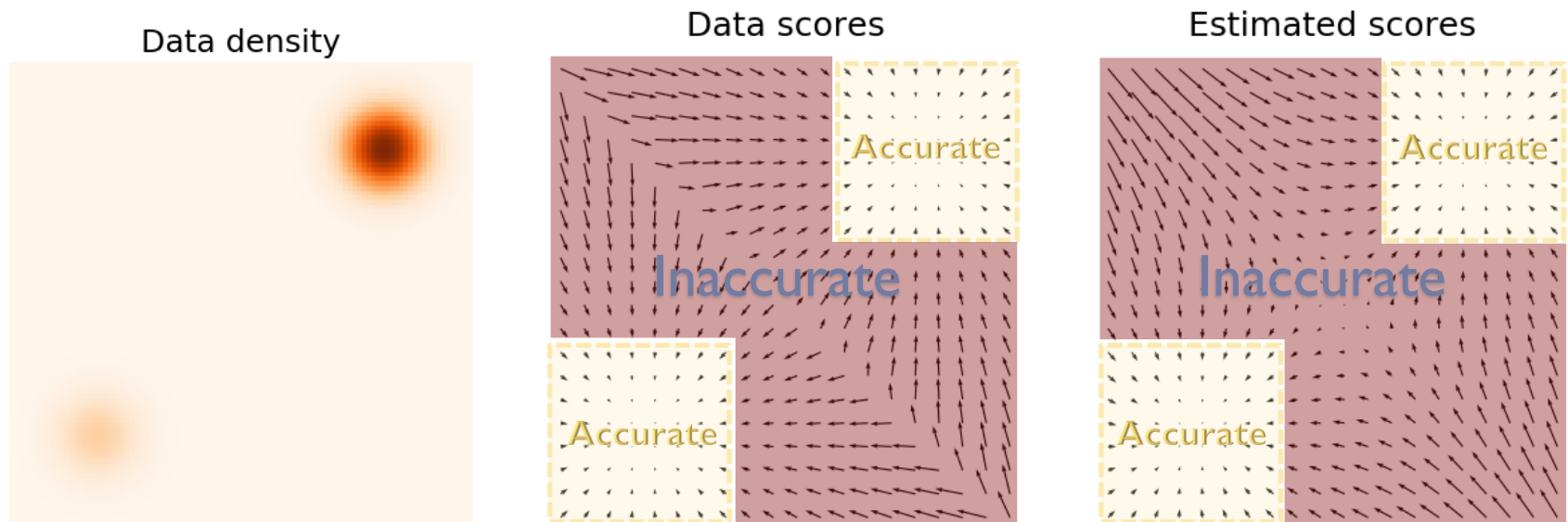
Gaussian perturbation

- The solution to all pitfalls: **Gaussian perturbation!**
- Manifold + noise
- Score matching on noisy data.



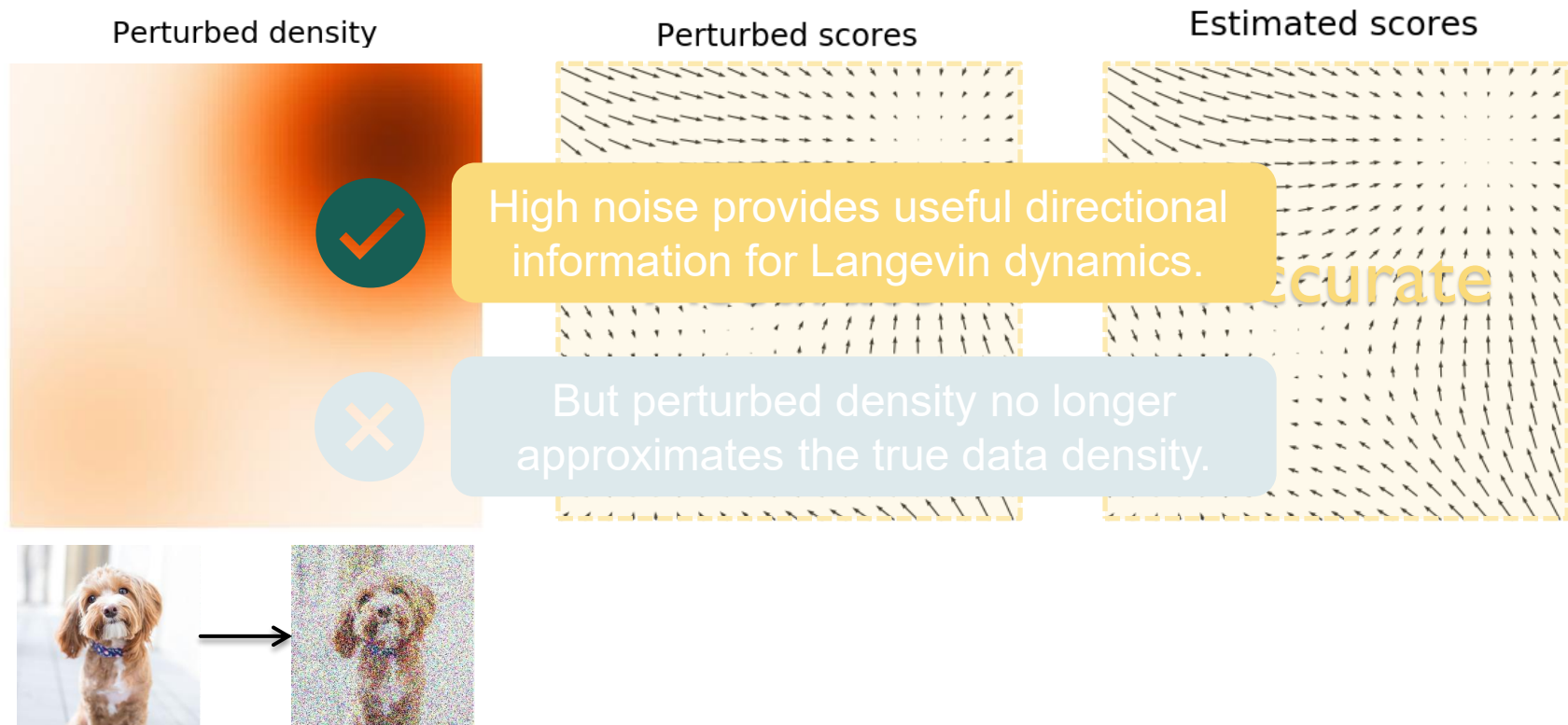
$$\mathcal{N}(0; 0.0001)$$

Challenge in low data density regions



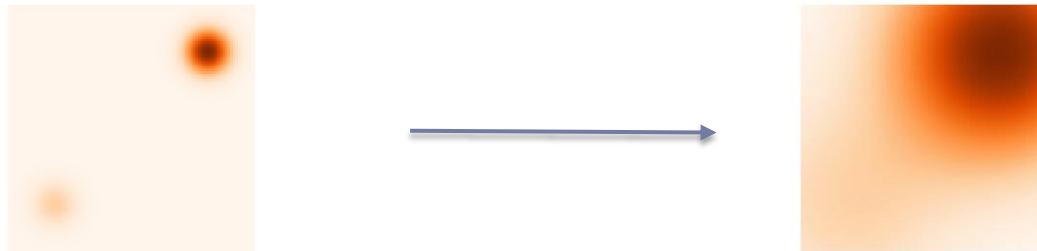
Song and Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution." NeurIPS 2019.

Improving score estimation by adding noise



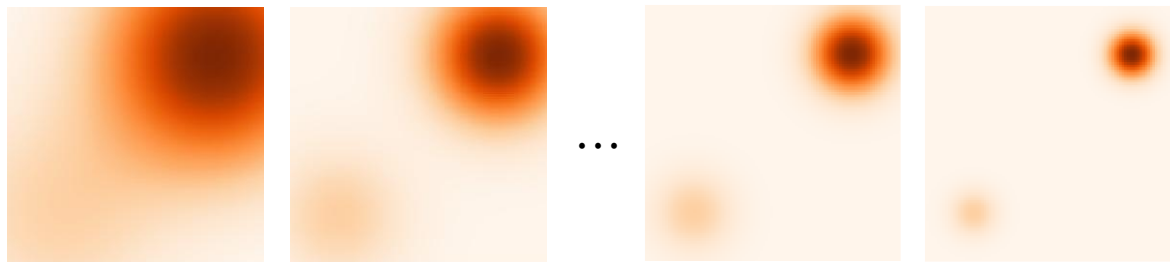
Multi-scale Noise Perturbation

- How much noise to add?

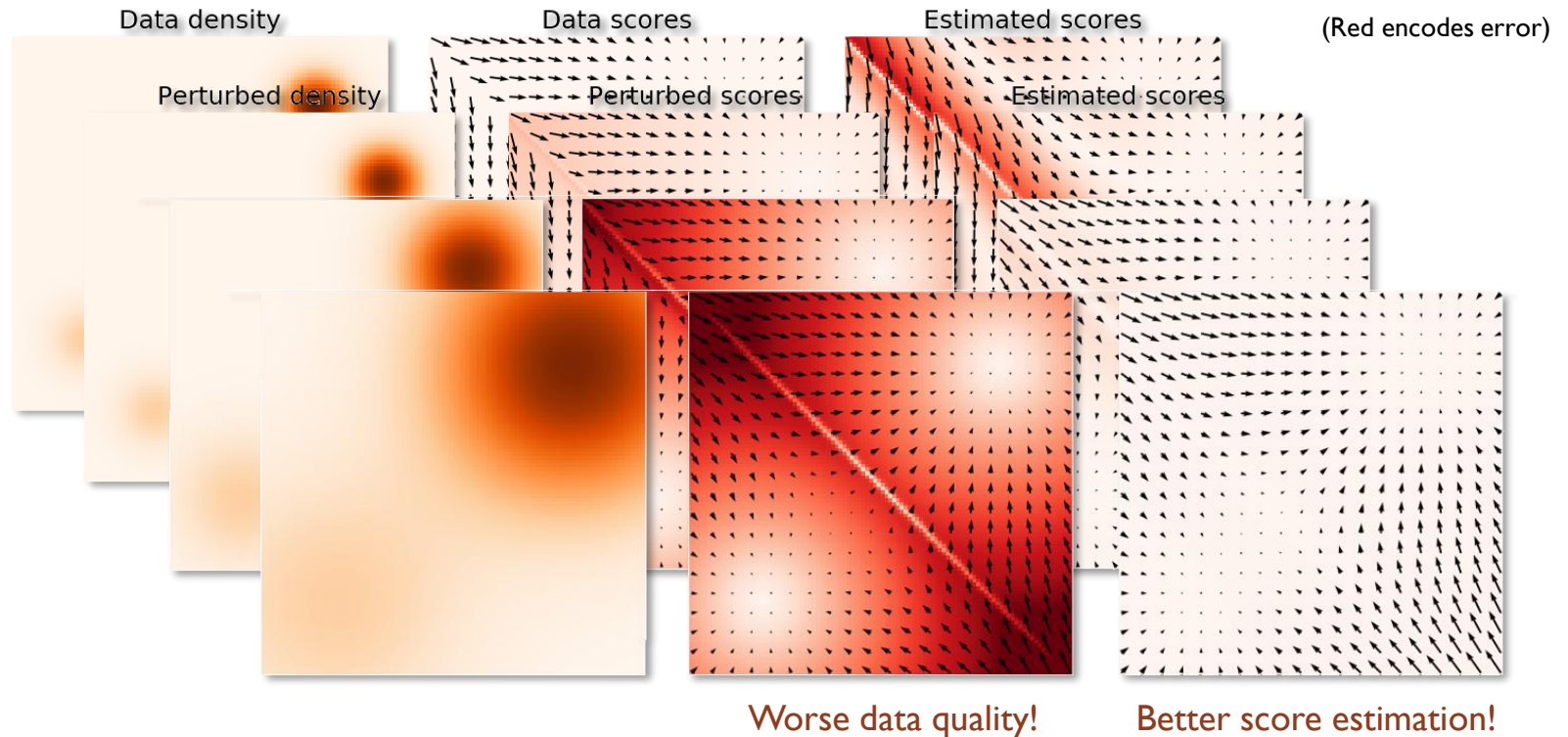


- Multi-scale noise perturbations.

$$\sigma_1 > \sigma_2 > \cdots > \sigma_{L-1} > \sigma_L$$

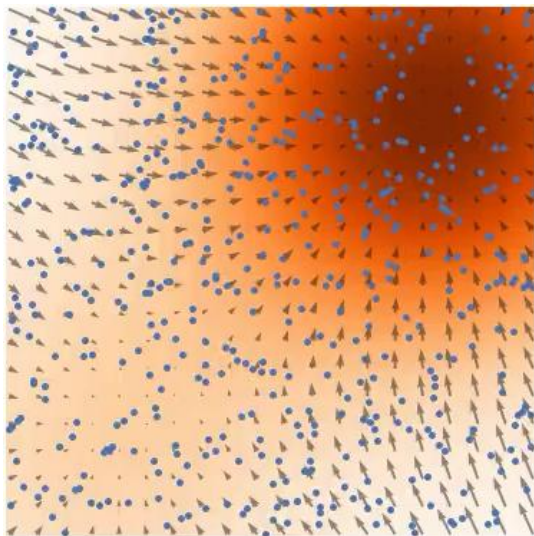


Trading off Data Quality and Estimation Accuracy

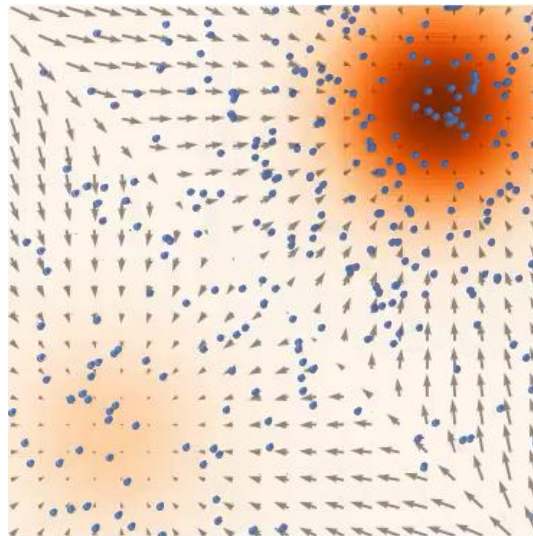


Annealed Langevin Dynamics: Joint Scores to Samples

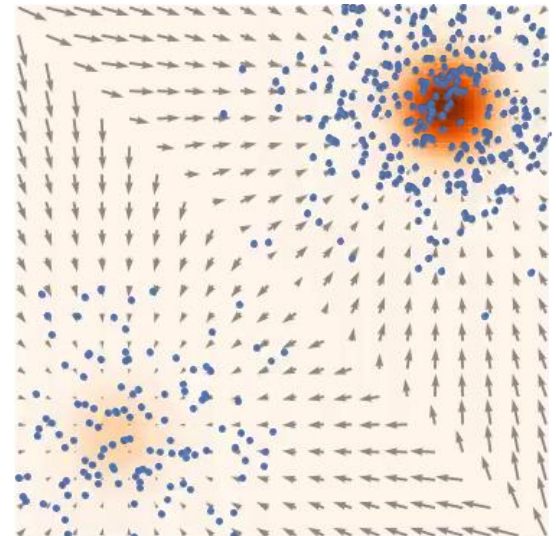
- Sample using $\sigma_1, \sigma_2, \dots, \sigma_L$ sequentially with Langevin dynamics.
- Anneal down the noise level.
- Samples used as initialization for the next level.



σ_1

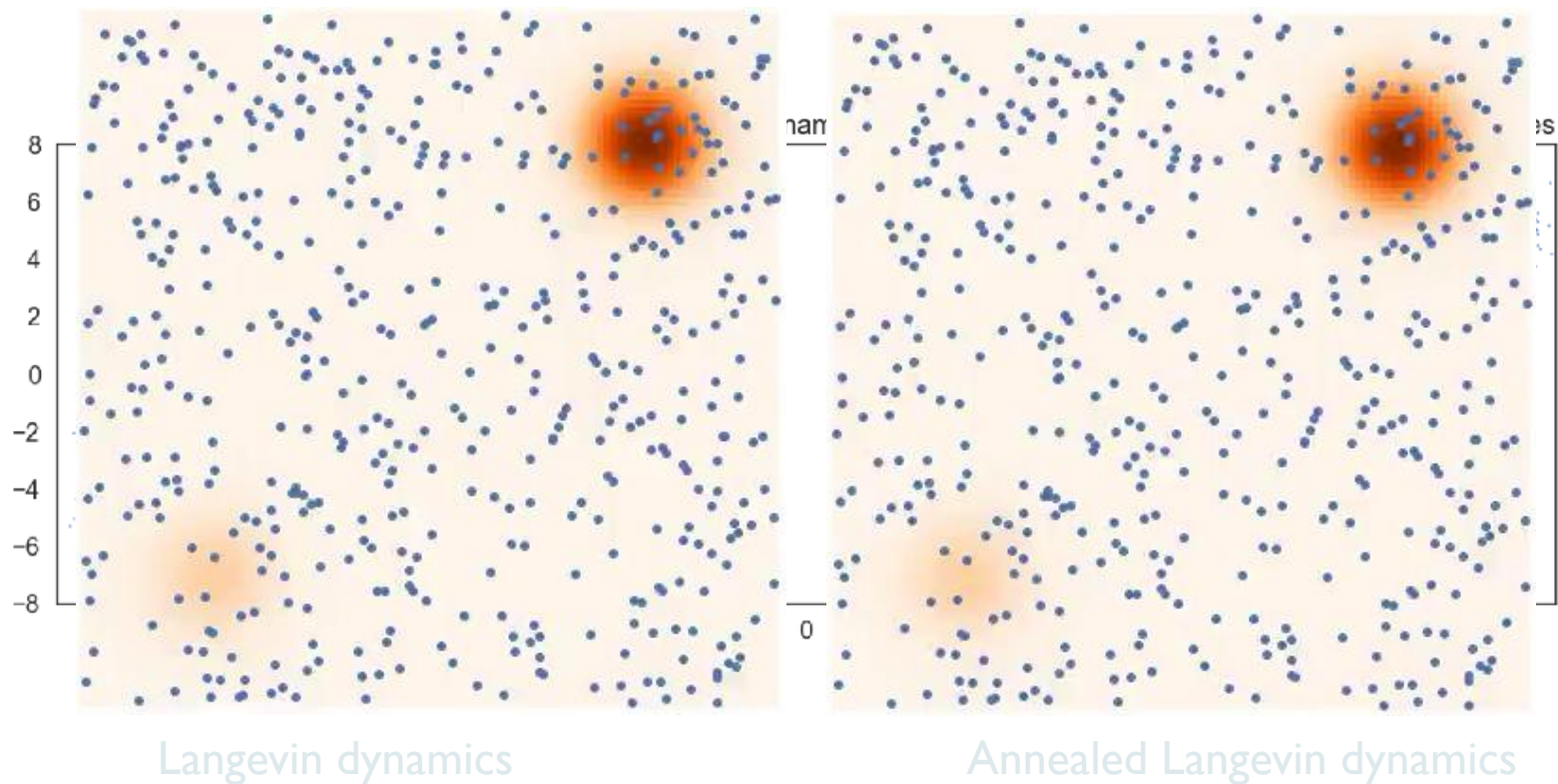


σ_2

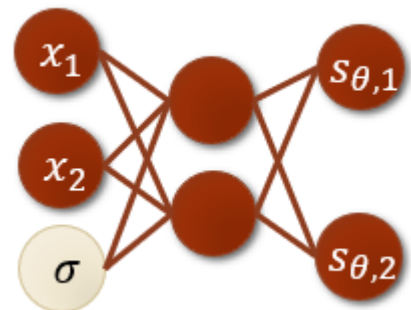
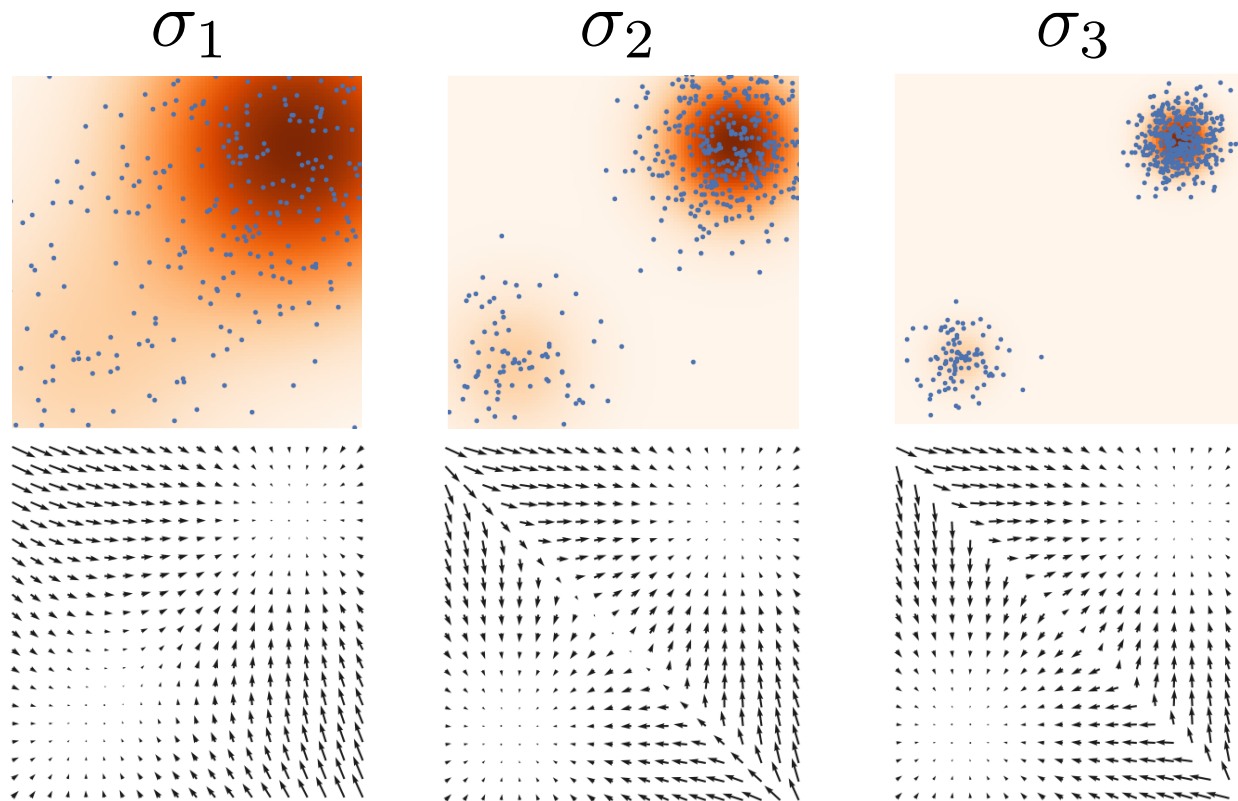


σ_3

Comparison to the vanilla Langevin dynamics



Joint Score Estimation via Noise Conditional Score Networks



Noise Conditional
Score Network
(NCSN)

Training noise conditional score networks

- Which score matching loss?
 - Sliced score matching?
 - Denoising score matching?
- Denoising score matching is naturally suitable, since the goal is to estimate the score of perturbed data distributions.
- Weighted combination of denoising score matching losses

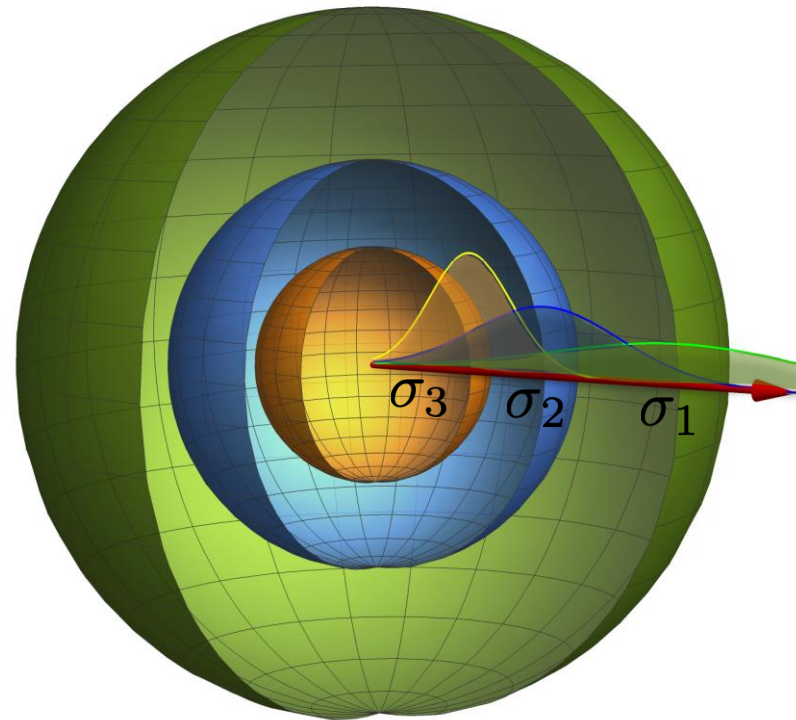
$$\begin{aligned} & \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) E_{q_{\sigma_i}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log q_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, \sigma_i)\|_2^2] \\ &= \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma_i}(\tilde{\mathbf{x}} | \mathbf{x}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}}, \sigma_i)\|_2^2] + \text{const.} \\ &= \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x} + \sigma_i \mathbf{z}, \sigma_i) + \frac{\mathbf{z}}{\sigma_i} \right\|_2^2 \right] + \text{const.} \end{aligned}$$

Choosing noise scales

- **Key intuition:** adjacent noise scales should have sufficient overlap to facilitate transitioning across noise scales in annealed Langevin dynamics.
- A geometric progression with sufficient length.

$$\sigma_1 > \sigma_2 > \sigma_3 > \cdots > \sigma_{L-1} > \sigma_L$$

$$\frac{\sigma_1}{\sigma_2} = \frac{\sigma_2}{\sigma_3} = \cdots = \frac{\sigma_{L-1}}{\sigma_L}$$



Choosing the weighting function

- Weighted combination of denoising score matching losses

$$\frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x} + \sigma_i \mathbf{z}, \sigma_i) + \frac{\mathbf{z}}{\sigma_i} \right\|_2^2 \right]$$

$$\lambda : \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$$

- How to choose the weighting function: $\lambda(\sigma_i) = \sigma_i^2$
- Goal:** balancing different score matching losses in the sum $\rightarrow$

$$\begin{aligned} & \frac{1}{L} \sum_{i=1}^L \sigma_i^2 E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x} + \sigma_i \mathbf{z}, \sigma_i) + \frac{\mathbf{z}}{\sigma_i} \right\|_2^2 \right] \\ &= \frac{1}{L} \sum_{i=1}^L E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \sigma_i \mathbf{s}_{\theta}(\mathbf{x} + \sigma_i \mathbf{z}, \sigma_i) + \mathbf{z} \right\|_2^2 \right] \\ &= \frac{1}{L} \sum_{i=1}^L E_{\mathbf{x} \sim p_{\text{data}}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \boldsymbol{\epsilon}_{\theta}(\mathbf{x} + \sigma_i \mathbf{z}, \sigma_i) + \mathbf{z} \right\|_2^2 \right] \quad [\boldsymbol{\epsilon}_{\theta}(\cdot, \sigma_i) := \sigma_i \mathbf{s}_{\theta}(\cdot, \sigma_i)] \end{aligned}$$

Training noise conditional score networks

- Sample a mini-batch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}$
- Sample a mini-batch of noise scale indices

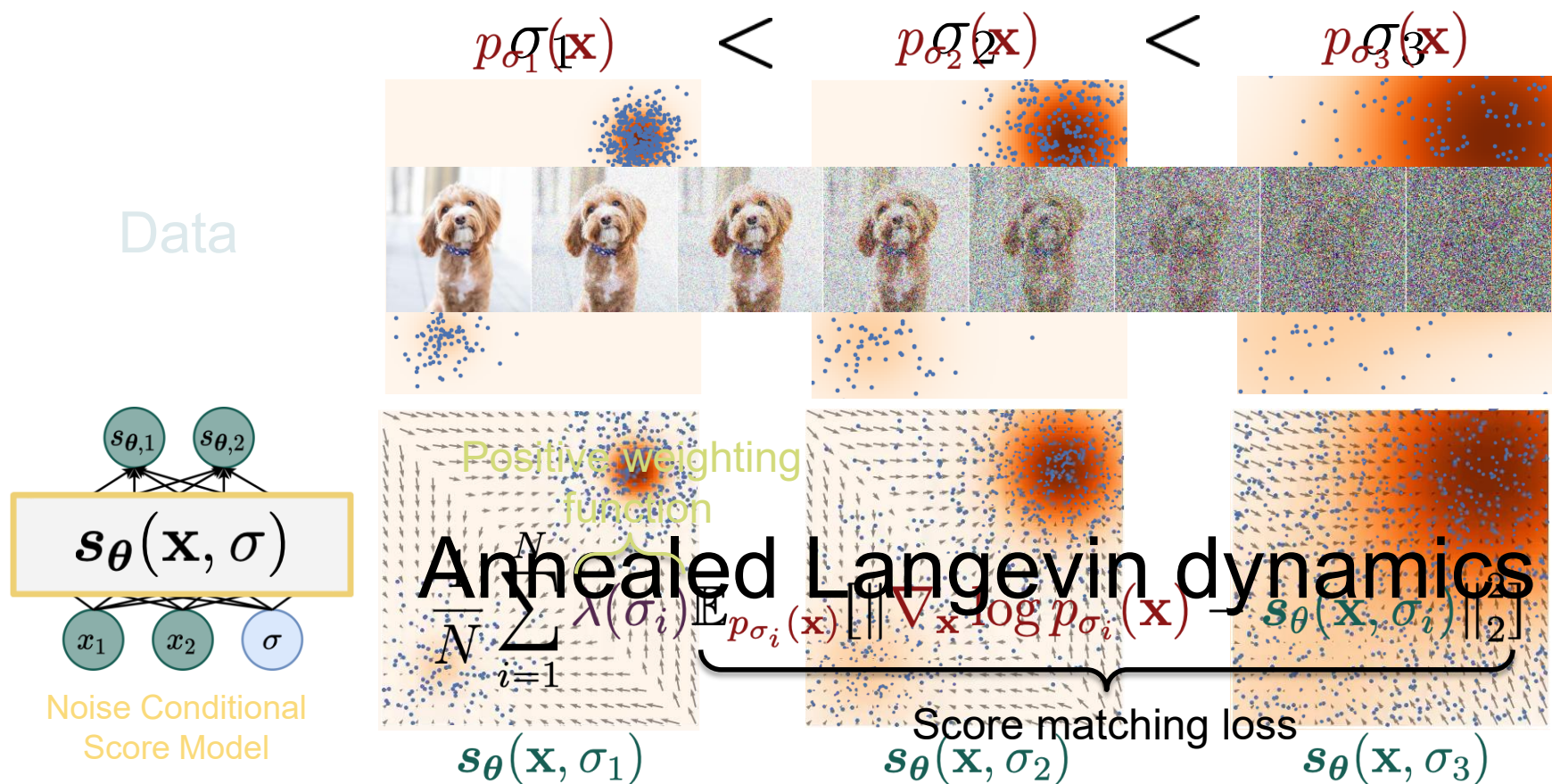
$$\{i_1, i_2, \dots, i_n\} \sim \mathcal{U}\{1, 2, \dots, L\}$$

- Sample a mini-batch of Gaussian noise $\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n\} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Estimate the weighted mixture of score matching losses

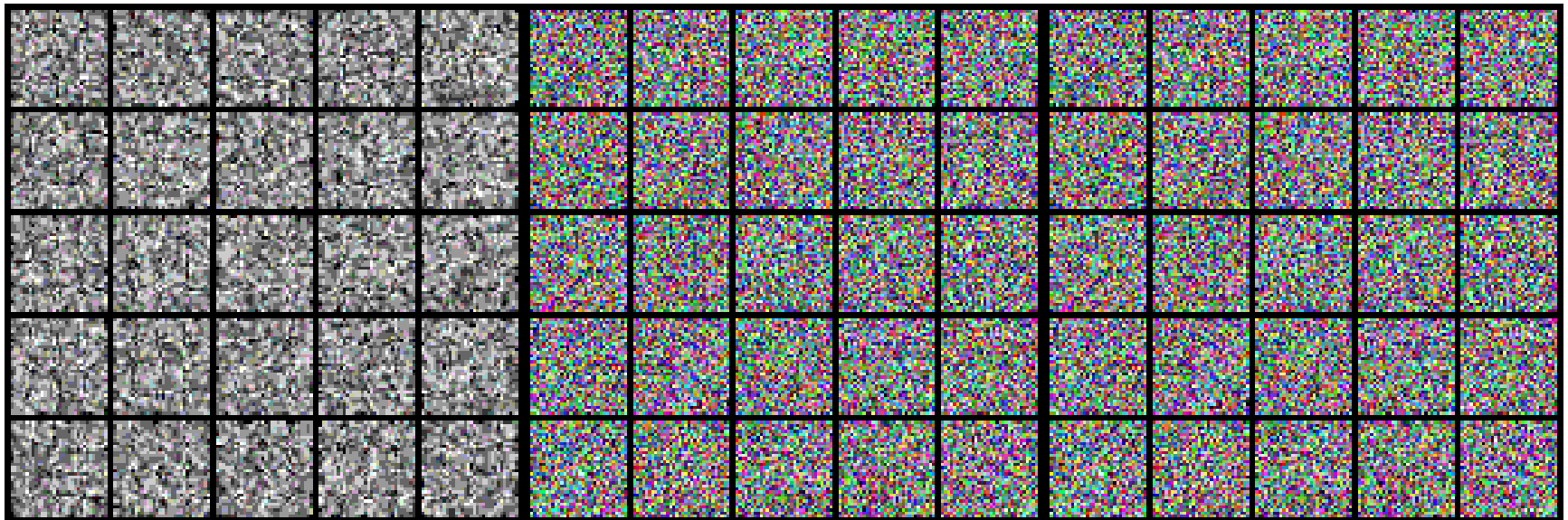
$$\frac{1}{n} \sum_{k=1}^n \left[\|\sigma_{i_k} \mathbf{s}_{\theta}(\mathbf{x}_k + \sigma_{i_k} \mathbf{z}_k, \sigma_{i_k}) + \mathbf{z}_k\|_2^2 \right]$$

- Stochastic gradient descent
- As efficient as training one single non-conditional score-based model

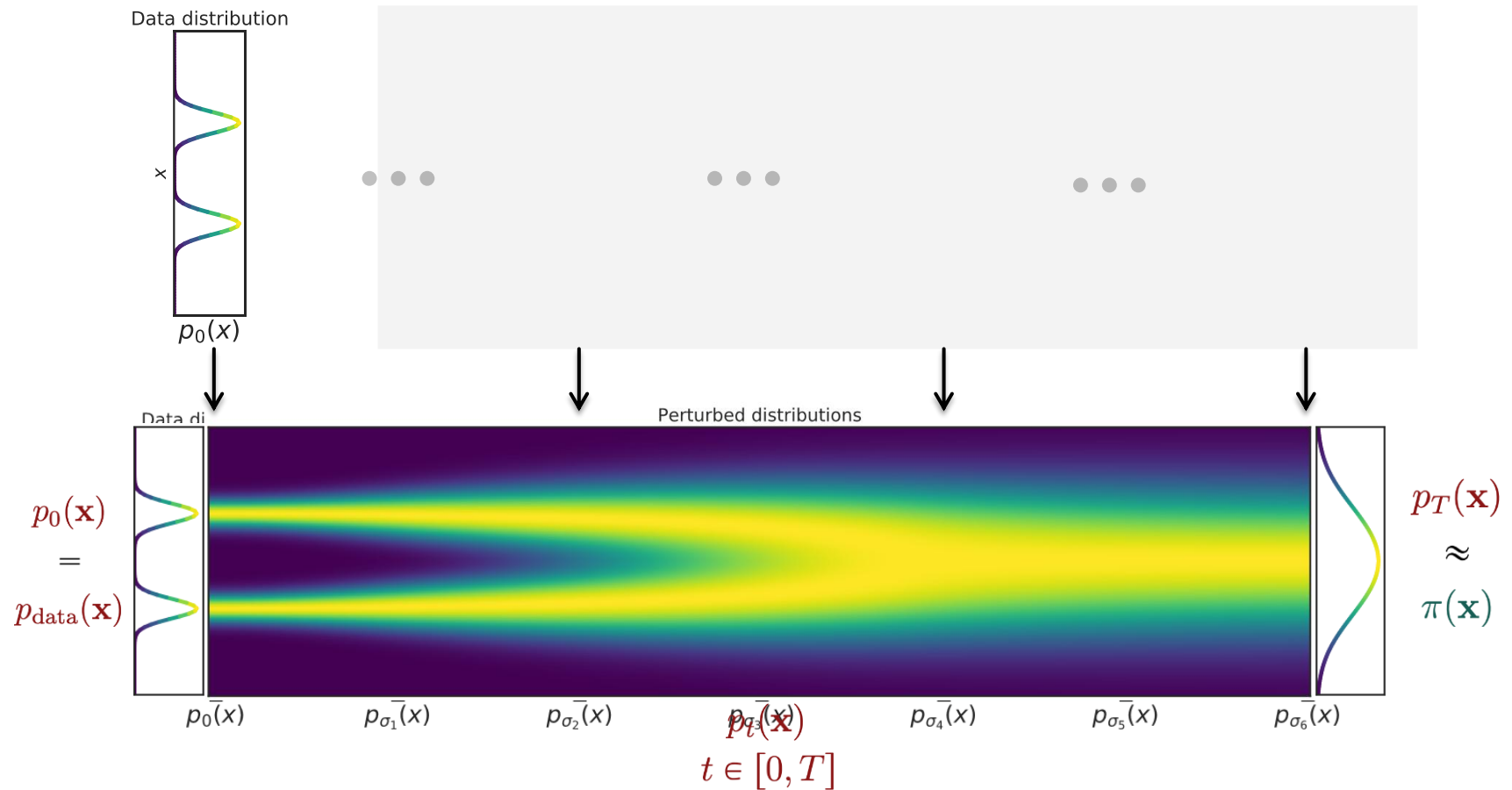
Using multiple noise levels



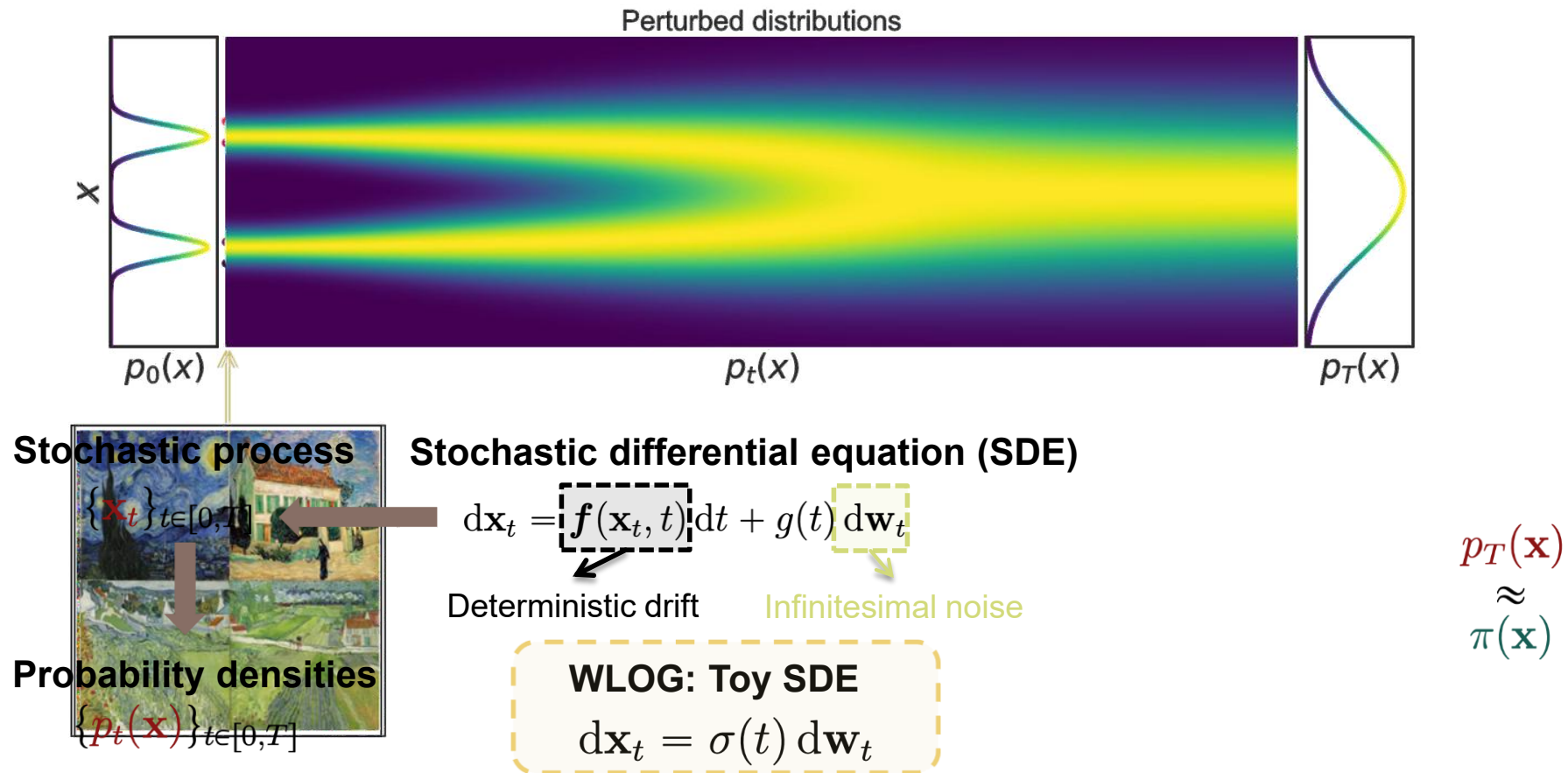
Experiments: Sampling



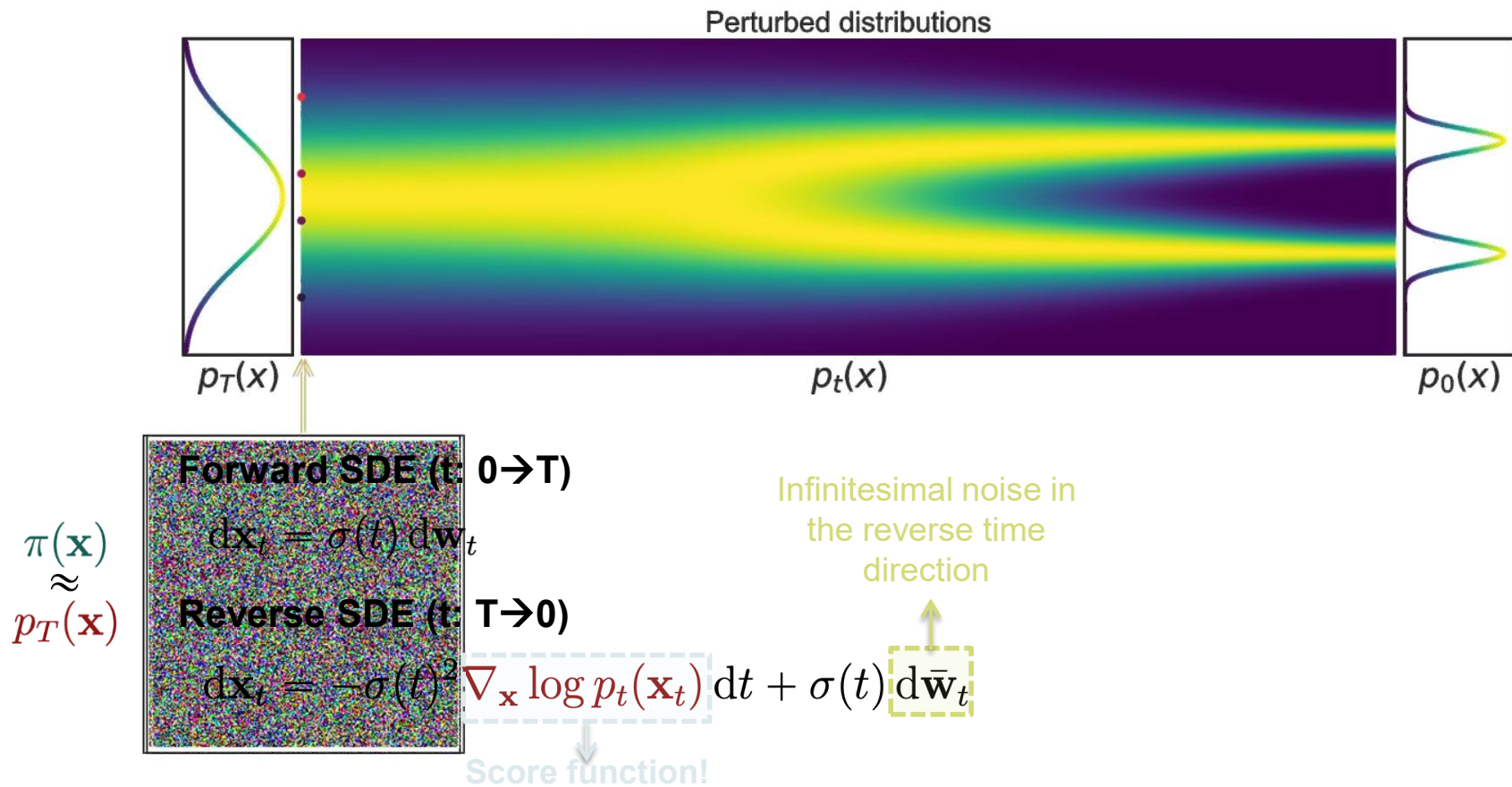
Infinite noise levels



Perturbing data with stochastic processes



Generation via reverse stochastic processes



Score-based generative modeling via SDEs

- Time-dependent score-based model

$$\mathbf{s}_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

- Training:

$$\mathbb{E}_{t \in \mathcal{U}(0, T)} [\lambda(t) \mathbb{E}_{p_t(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - \mathbf{s}_\theta(\mathbf{x}, t)\|_2^2]]$$

- Reverse-time SDE

$$d\mathbf{x} = -\sigma^2(t) \mathbf{s}_\theta(\mathbf{x}, t) dt + \sigma(t) d\bar{\mathbf{w}}$$

- Euler-Maruyama (analogous to Euler for ODEs)

$$\begin{aligned} \mathbf{x} &\leftarrow \mathbf{x} - \sigma(t)^2 \mathbf{s}_\theta(\mathbf{x}, t) \Delta t + \sigma(t) \mathbf{z} \quad (\mathbf{z} \sim \mathcal{N}(\mathbf{0}, |\Delta t| \mathbf{I})) \\ t &\leftarrow t + \Delta t \end{aligned}$$

Predictor-Corrector sampling methods

- Predictor-Corrector sampling.
 - **Predictor:** Numerical SDE solver
 - **Corrector:** Score-based MCMC

